



Lecture #2.1

Introduction to Part II

MAS418

Programming for Intelligent Robotics and Industrial systems

Part II: PLC Software Development

Autumn 2025

Daniel Hagen, PhD



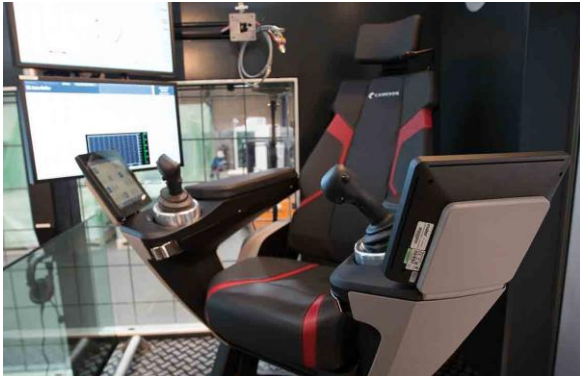
Associate Professor **Daniel Hagen**

- 37 years old, from **Lyngdal**, lives in **Arendal** with my wife, her 16-year-old daughter, and our five-year-old son
- **BSc** (2012) and **MSc** (2014) in **Mechatronics** from **UiA** based on my trade certificate as an **Automation Mechanic** (2009)
- Received my **PhD** with a Specialization in **Mechatronics** in 2020
- Between **MSc** and **PhD** I worked in **Cameron Sense** with PLC-based Control Systems (Siemens) for oil drilling equipment
- After the **PhD** I worked in **Red Rock AS** as a **Senior R&D Engineer** responsible for the PC based Control Systems (Beckhoff) for cranes
- Since **March 2022** I have worked full-time at **UiA**: MAS418, MAS514, MAS518
- **Director** of **UiA Motion Lab** and **Head** of the **Intelligent Mechatronics** research group (**iTron**)

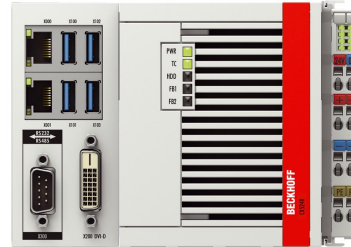
Background

Operator/Human-in-the-loop Control

Human Machine Interface (HMI)



Real-Time Controller (PLC)



Mechatronic Application



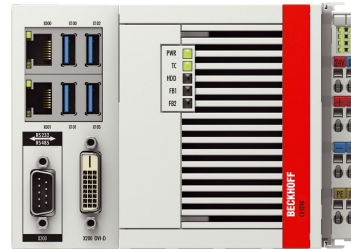
Motivation

Autonomous Control Systems



Top-level control

Real-Time Controller (PLC)



Low-level (machine) control

Mechatronic Application

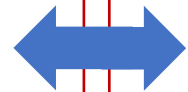


MAS418 Overview

Distributed Control System

Part III – Project Work in Groups

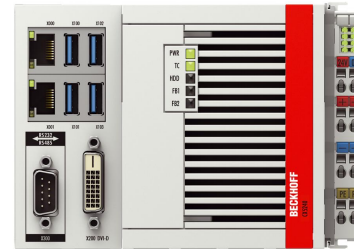
Part I



Top-level control – motion reference, data logging, visualization, etc.

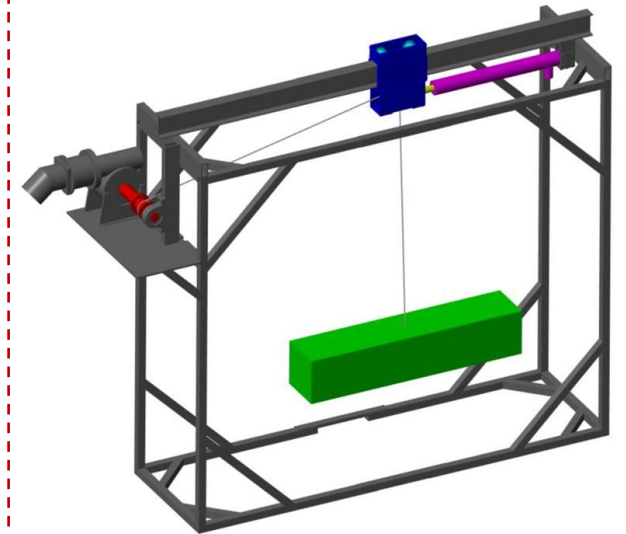
Part II

Real-Time Controller (PLC)



Low-level (machine) control – safety, open-loop (manual) and closed-loop (automatic) motion control

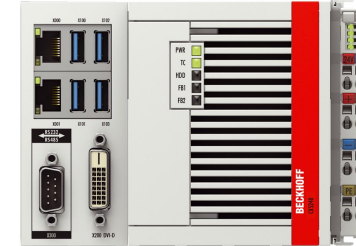
Mechatronic Application



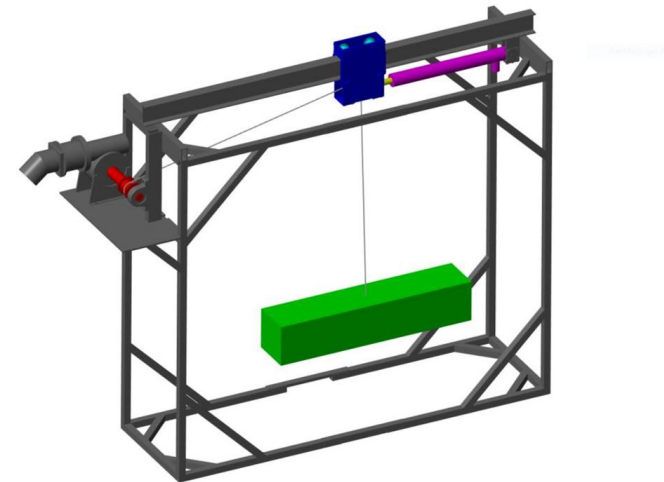
Part II Overview

- **PLC programming** using **Structured Text (ST)** in **Beckhoff TwinCAT** software
- Focus on **object-oriented programming** and **version control**
- **Programming** of a typical **industrial mechatronic** application (machine) **control system**
- **Testing** of the program in PLC simulator

Real-Time Controller (PLC)



Mechatronic Application



Information

Teacher - **Part II**

Daniel Hagen

Email: daniel.hagen@uia.no

Office: D3-037

Unofficial [Discord server](#) for support

In **Part III** both Daniel and Kristian are involved

Updated information and course materials
available in Canvas

Electives

MAS418

Programming for Intelligent Robotics and Industrial systems

7.5 stp

MAS509

Biomedical Signal Processing and Instrumentation

7.5 stp

MAS511

Bio-mechatronics

7.5 stp

MAS512

Computer Vision and Embedded AI

7.5 stp

MAS514

Introduction to Autonomous Robotics

7.5 stp

MAS515

Mechanical Design of Mechatronic Systems

7.5 stp

PRA400

Internship for master's students in engineering and science

15 stp

Activities

• Lectures

- Five main lectures (week 34, 36, 38, 40, 43)
- **Fridays** 08:15-10:00 (D3-053)

• Individual Lab Exercises

- Five lab exercises (week 35, 37, 39, 41, 44)
- **Wednesdays** 08:15-12:00 (D2-024)

• Project work in Groups

- Lab work related Part III (week 46-47)
- **Wednesdays** 08:15-12:00 (D2-024)

• Homework

- Complete exercises that are not finished during the lab exercise
- 5-7 hours a week is expected

Change of the plan will be announced in [Canvas](#)!

Uke 35

	Mandag 25/8	Tirsdag 26/8	Onsdag 27/8	Torsdag 28/8	Fredag 29/8
8					
9			Lab.-undervisning Lab.-undervisning og prosjekt 08:15 - 12:00 D. Hagen K.M. Knausgård D D2 024 (32 pl.)		Forelesning Forelesning dobbelttime 08:15 - 10:00 D. Hagen
10					
11					
12					
13					
14					
15					

Juli 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
27		1	2	3	4	5	6
28	7	8	9	10	11	12	13
29	14	15	16	17	18	19	20
30	21	22	23	24	25	26	27
31	28	29	30	31			

August 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
31					1	2	3
32	4	5	6	7	8	9	10
33	11	12	13	14	15	16	17
34	18	19	20	21	22	23	24
35	25	26	27	28	29	30	31

September 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
36	1	2	3	4	5	6	7
37	8	9	10	11	12	13	14
38	15	16	17	18	19	20	21
39	22	23	24	25	26	27	28
40	29	30					

Oktober 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
40			1	2	3	4	5
41	6	7	8	9	10	11	12
42	13	14	15	16	17	18	19
43	20	21	22	23	24	25	26
44	27	28	29	30	31		

November 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
44						1	2
45	3	4	5	6	7	8	9
46	10	11	12	13	14	15	16
47	17	18	19	20	21	22	23
48	24	25	26	27	28	29	30

Desember 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
49	1	2	3	4	5	6	7
50	8	9	10	11	12	13	14
51	15	16	17	18	19	20	21
52	22	23	24	25	26	27	28
	29	30	31				

Self-study

Part #1

Part #2

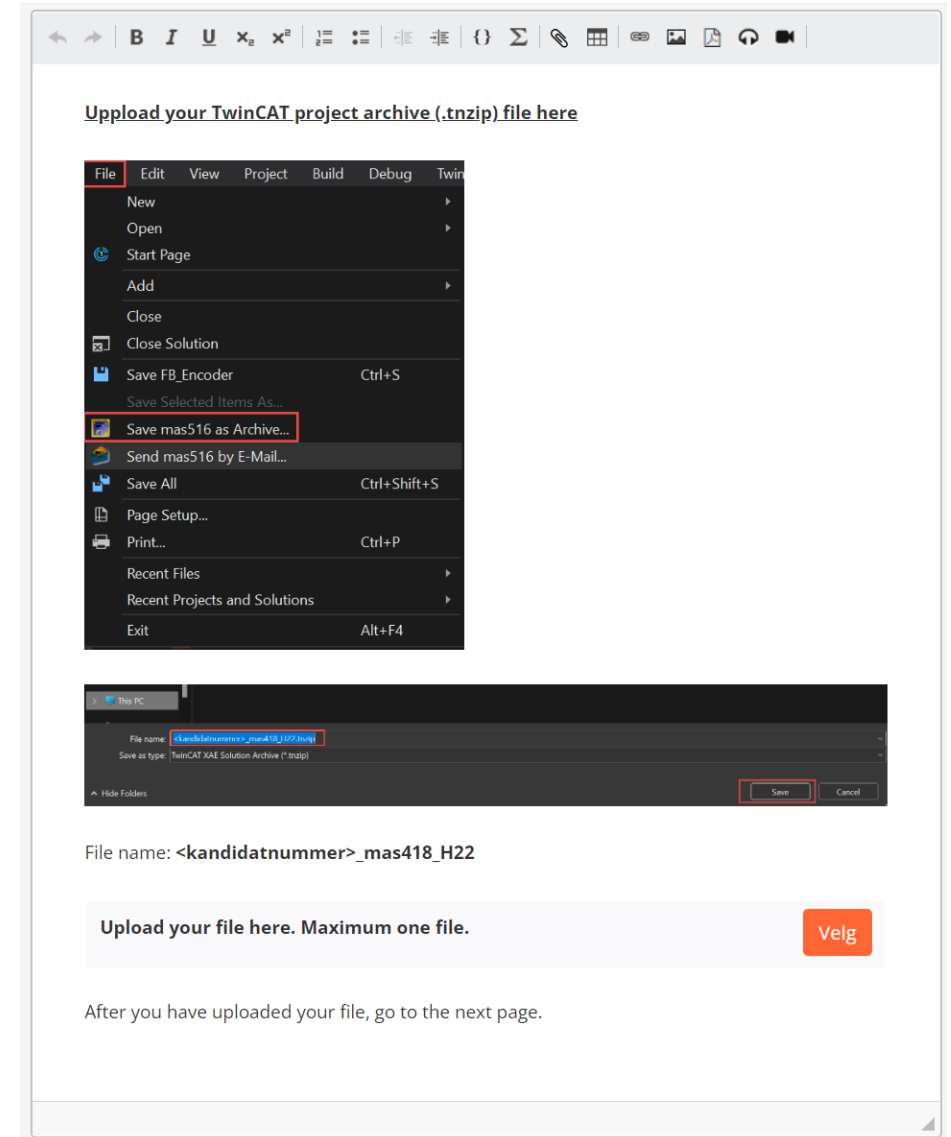
Part #3 (project)

Part Exam

Activities

• Examination Part II

- PLC programming task in TwinCAT on lab PCs
- 3 hours - **50%** of total grade
- Submission of .tnzip file in Canvas assignment
- Grading (100%):
 - Correct and sensible use of **Enumeration** (10%)
 - Correct and sensible use of **Structure** (10%)
 - Correct and sensible use of **Program** (10%)
 - Correct and sensible use of **Function** (10%)
 - Correct and sensible use of **Instruction** (CASES / IF-ELSE statements) (10%)
 - Correct and sensible use of **Function block** in an object-oriented way (30%)
 - Reuse of a **Function block** (10%)
 - Use of **Method** (10%)
 - Use of **Interface** (10%)
 - A working **simulator** demonstrating that the code works as intended (20%)



Literature

- **Optional Book: PLC Controls with Structured Text (ST), V4**
 - <https://www.adlibris.com/nb/bok/plc-controls-with-structured-text-st-v4-9788776919733>
- **Beckhoff Information System**
 - https://infosys.beckhoff.com/index_en.htmLinks to an external site.
- **CODESYS Identifier Designation**
 - https://content.helpme-codesys.com/en/CODESYS%20Development%20System/_cds_identifier_s.html
- **PLC programming using TwinCAT 3**
 - <https://www.youtube.com/playlist?list=PLimaF0nZKYHz3I3kFP4myaAYjmYk1SowOL>Links to an external site.
- **TwinCAT 3 Tutorial**
 - <http://www.contactandcoil.com/twincat-3-tutorial/> Links to an external site.
- **Learning PLCs with Structured Text**
 - https://www.youtube.com/playlist?list=PLE1CU6EebvTCJCMIUOSWgMs_eMaW-2k5zH

Overview

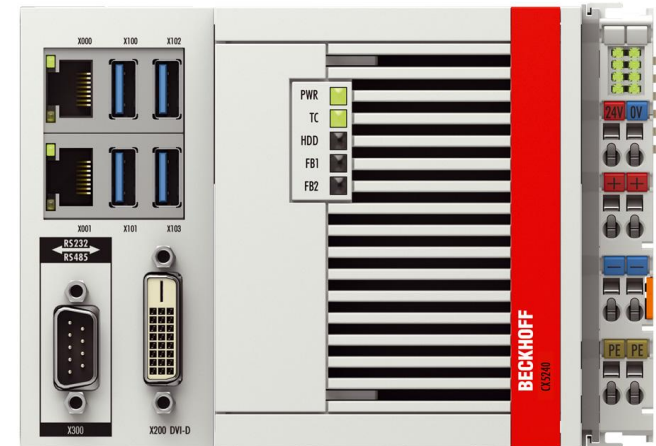
Introduction

Part I: Introduction to PLC programming

Part II: TwinCAT basics

Part III: Data types & Std. library

Summary

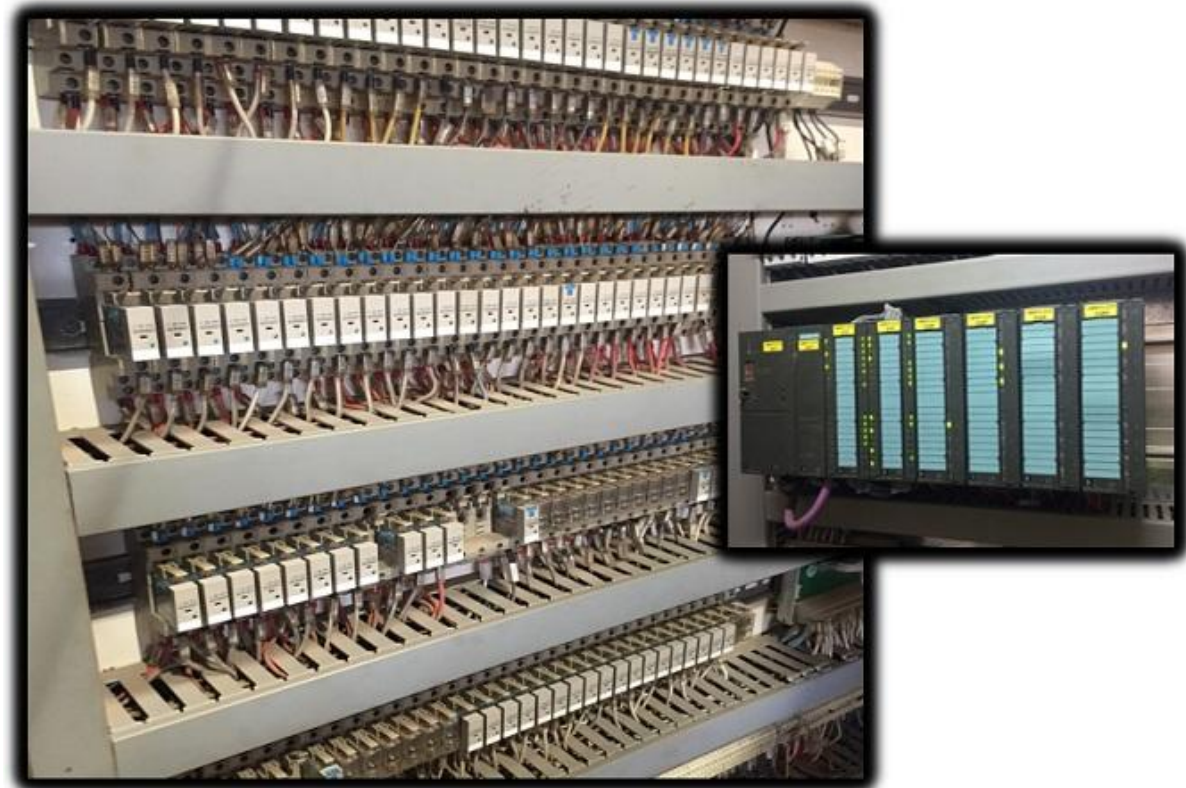


Part I: Introduction to PLC programming

1. History
2. IEC61131-3 standard
3. Version control

History - PLC

- Before **PLCs**, it was possible to automate a process using relay-based control systems
 - Requires a huge number of relays, cabling and space → difficult to maintain
 - Small change in system require rewiring
 - Single wire brake could cause the whole system to not work
- In the end of the **1950s** the first ideas for replacement of these hard-wired relay systems arrived
 - Independently the companies Allen-Bradley, GM and Siemens investigated the electronic replacement, and they created the first PLCs



<https://ladderlogicworld.com/relay-logic-vs-ladder-logic/>

History - PLC

- In **1973**, the first recognizes mass-marked PLC to achieve success arrived (**Modicon 184**)
- **Siemens PLC history:**
 - **1958:** SIMATIC Version G
 - **1973:** SIMATIC S3
 - **1979:** SIMATIC S5
 - **1995:** SIMATIC S7
 - **2010:** Total Integrated Automation (TIA) portal
 - **2024:** SIMATIC AX

Siemens SIMATIC S-7 1500 (2012)



Modicon 184 (1973)



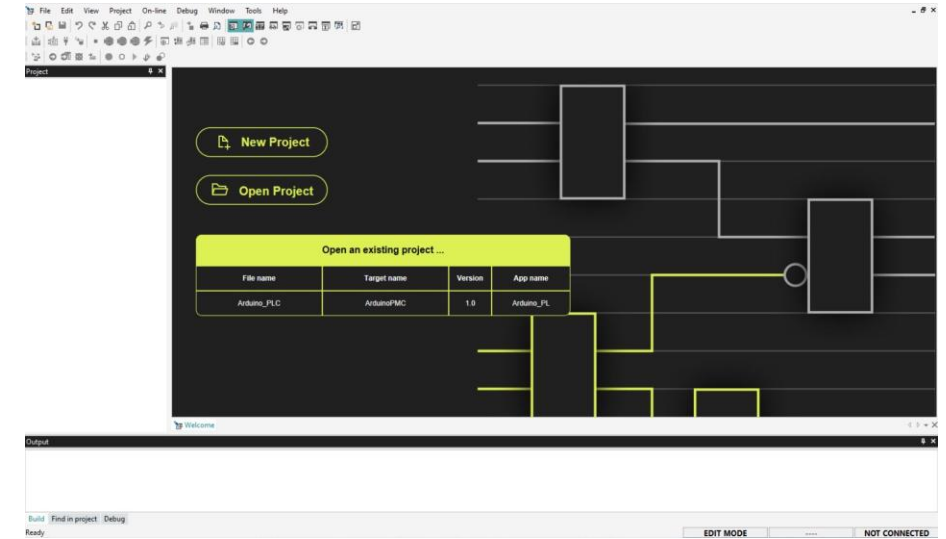
Siemens SIMATIC S-7 300 (1995)



History - Arduino PLC

- In **2022** Arduino released their own PLC IDE, allowing you to program the Arduino [Portenta Machine Control](#) using the 5 programming languages defined by the IEC 61131-3 standard
- In **2023** Arduino released their own PLC hardware – [Arduino Opta](#), a microPLC with Industrial IoT Capabilities.

Arduino PLC IDE



<https://www.arduino.cc/pro/software-plc-ide>

Arduino Opta



<https://www.arduino.cc/pro/hardware-arduino-opta>

<https://www.youtube.com/watch?v=mq1xgubaQeA>

History - PC-based control (TwinCAT)

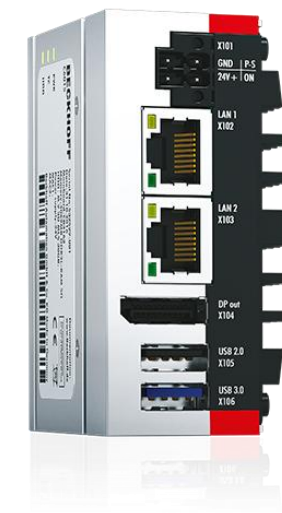
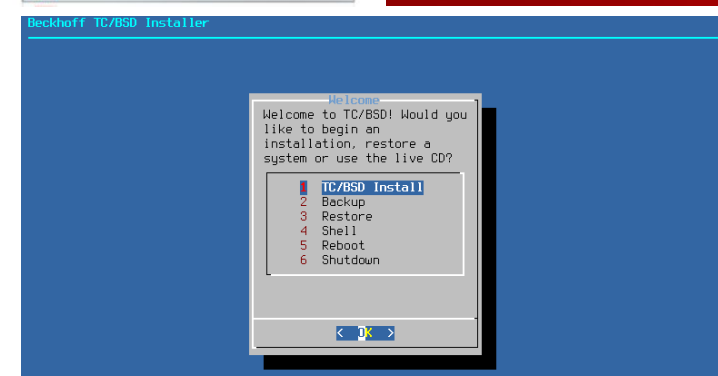
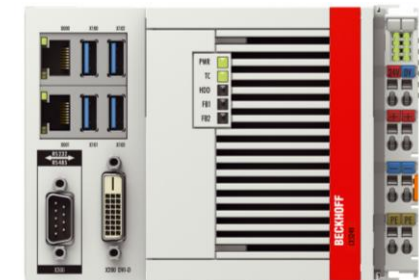
- The adoption of PCs in **1980s** was the start of interfacing PCs with PLCs
 - During **1990s** PLC monitoring software arrived for doing troubleshooting by technicians and that was the start of PC-based HMIs communication with PLC
- In **1980s** Beckhoff started with PC-based control running on standard PC in DOS
 - In **1986** the first industrial PLC was released
- In **1996** Beckhoff released **The Windows Control and Automation Technology (TwinCAT) 2**, where it was possible to turn a standard PC running Windows into a real-time capable controller (More details in Part 2)

Beckhoff C-4000 (1986)



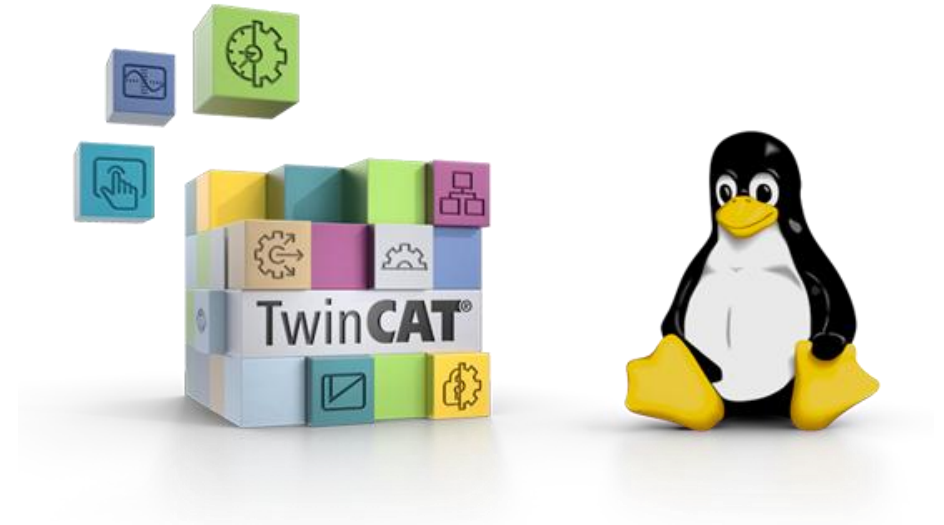
History - PC-based control (TwinCAT)

- In **2012** TwinCAT 3 was released where Beckhoff integrated both the development environment and the functionality of the system manager into MS Visual Studio where everything related to configuration, writing code, compiling, publishing, and doing version control could be done from this single **I**ntegrated **D**evelopment **E**nvironment (**IDE**)
- In **2020** there was released a TwinCAT runtime for Beckhoffs FreeBSD called Tc/BSD
 - However, all PLC software development is carried out in a Windows environment (or through virtual machine in Linux/OSx)



History - PC-based control (TwinCAT)

- In **2025** there is estimated marked release in 3rd quarter for [Linux®-based real-time control with TwinCAT](#)
- In **2026** there is estimated marked release in 3rd quarter for [TwinCAT PLC++](#)
- Other **upcoming** features:
 - TE1700 | [TE1700 | TwinCAT 3 CoAgent | Beckhoff Worldwide](#)
 - TE3850 | [TwinCAT 3 Machine Learning Creator](#)



IEC61131-3 standard

International standard for PLC programming

- One of the most significant milestones in PLC history was the introduction of the International Electrotechnical Commission (IEC) specification in **1982**
- It was the standard by which PLC software being developed was to be held against. It became published in **1993** as IEC 61131 International Standard for Programmable Controllers
- The introduction of the **IEC 61131-3 (2013)** was necessary as it brought consistency to all the software products on the market
 - This allows engineers and technicians to easily understand logic and program flow from any PLC software

1. Sequential Function Charts (SFC)

2. Ladder Diagram (LD)

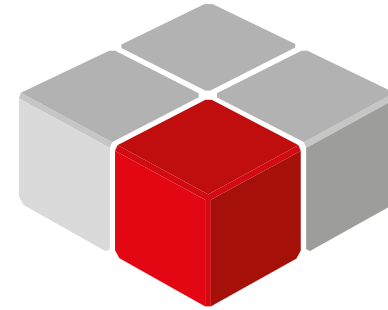
3. Function Block Diagram (FBD)

4. Instruction List (IL)

5. Structured Text (ST)

IEC61131-3 standard

- To be able to do PLC software in Visual Studio, Beckhoff relies on a compiler made by **CODESYS**
 - German manufacturer of IEC61131-3 automation software use by several PLC manufacturer e.g., Rexroth, WAGO, etc.
 - There is even a runtime for Raspberry Pi!
 - If you have worked with CODESYS you will find many similarities when using TwinCAT eXtended Automation Engineering (XAE) for PLC software development

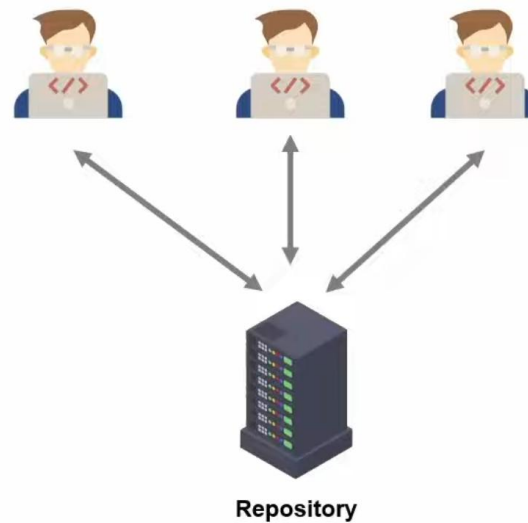
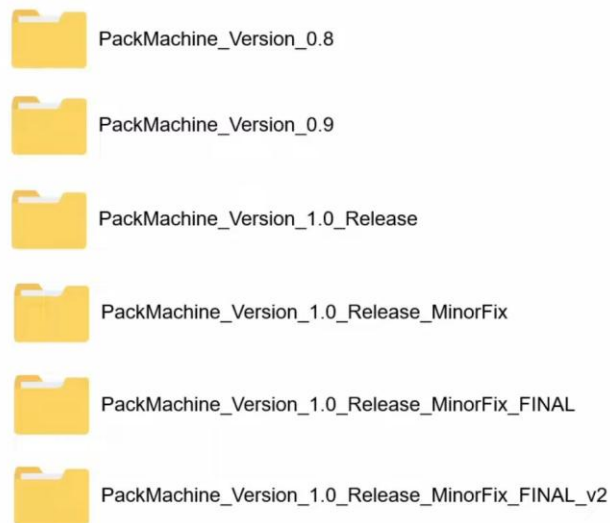


CODESYS

<https://www.codesys.com/>

Version control

- **Why do version control?**
 - Long-term history for every file
 - Branching and merging
 - Traceability



● CVS

● SVN

● **Git**

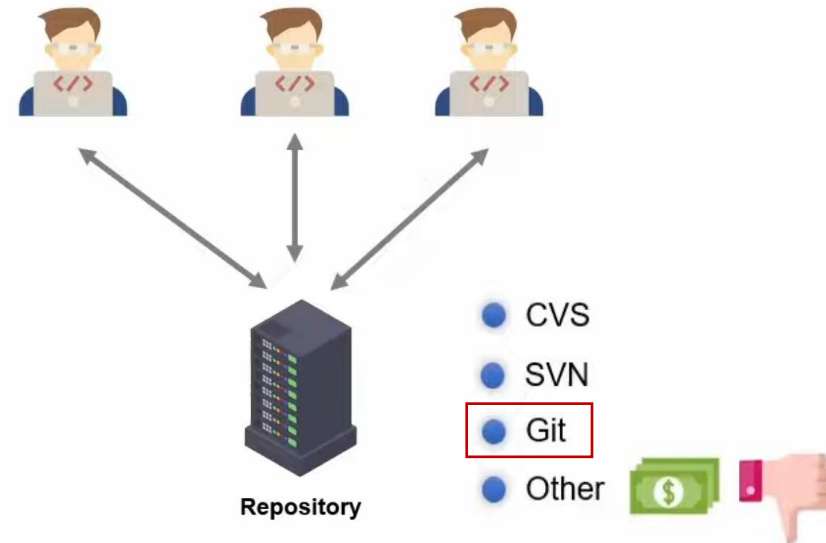
● Other



Version control

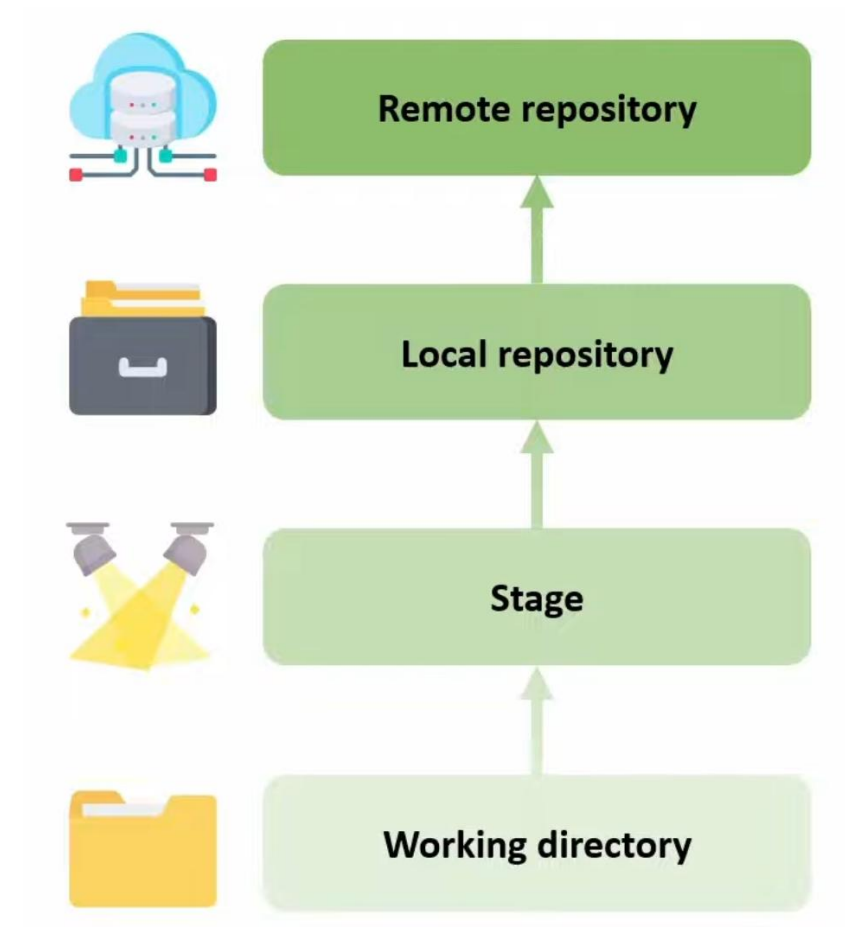
- **Git**

- The worlds most popular version control system
- A Distributed version control system and differs from traditional centralized version control system (SVN)
- Uses multiple repositories including a centralized repository and server, as well as some local repositories
- Does not rely on a central server
 - Can work locally (offline)
- The central server can be self hosted or using third-party hosting service such as **GitHub**, GitLab, etc.



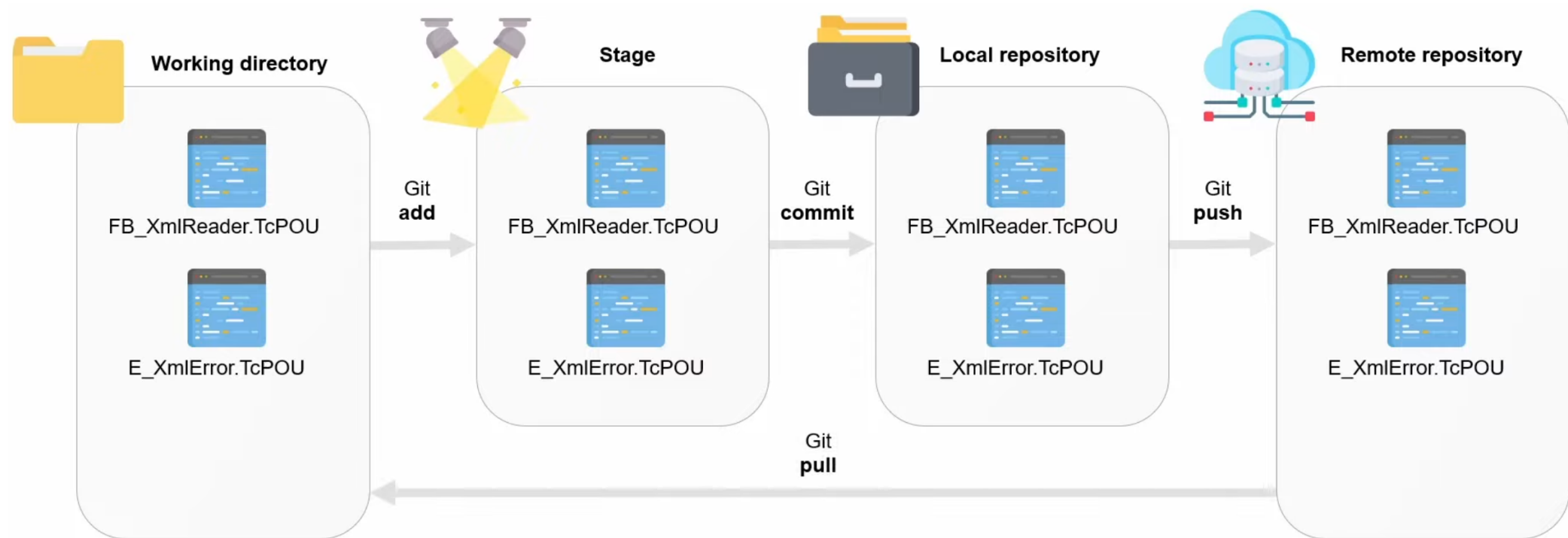
Version control

- **Git**
 - The main components



Version control

- **Git**
 - Workflow



Version control

- **Git**
 - **Git ignore** (.gitignore)
 - TwinCAT - <https://github.com/github/gitignore/blob/main/TwinCAT3.gitignore>
 - VisualStudio - <https://github.com/github/gitignore/blob/main/VisualStudio.gitignore>



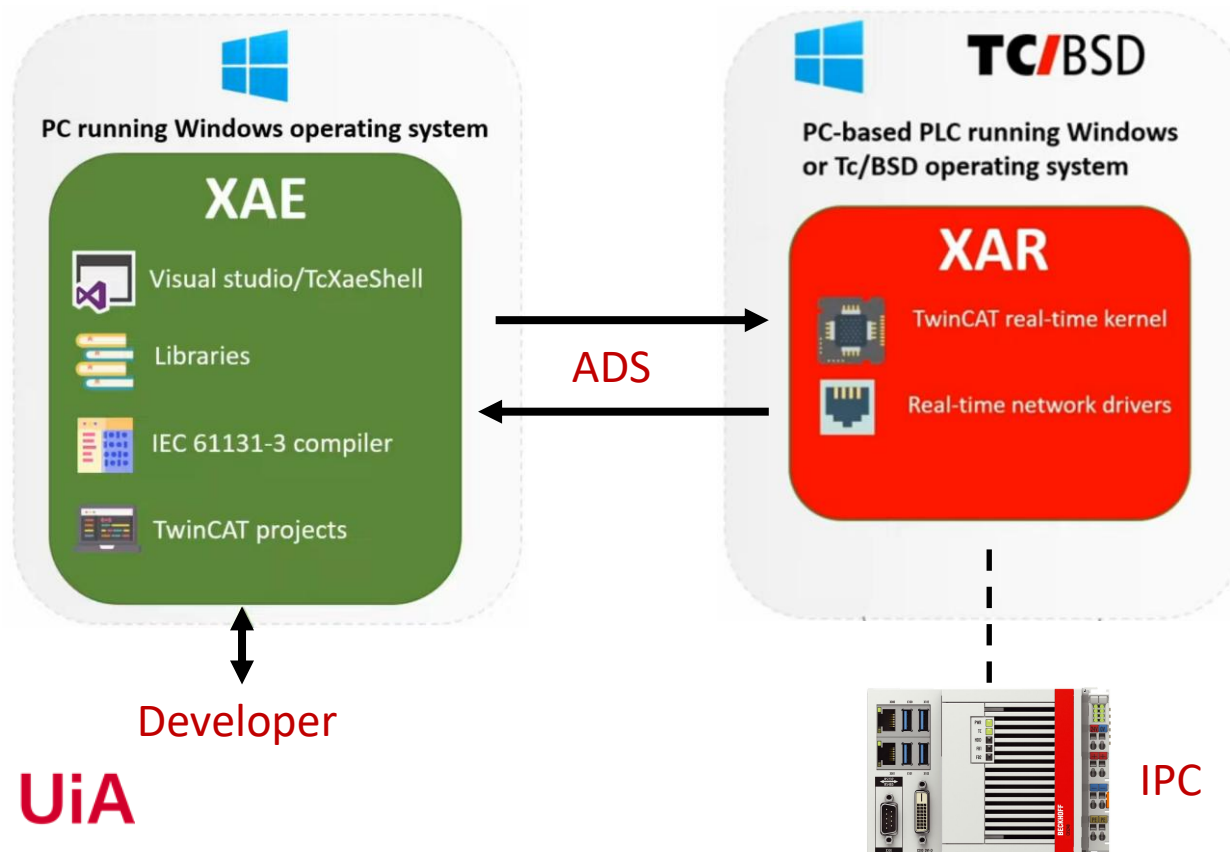
Part II: TwinCAT basics

1. Introduction to TwinCAT
2. Difference to SW running in OS

Introcution to TwinCAT

Consist of two main parts:

- TwinCAT eXtended Automation Engineering (XAE)
- TwinCAT eXtended Automation Runtime (XAR)



XAE:

- IDE based on Visual Studio
- Libraries for various functionalities
- The IEC61131-3 compiler itself
- Used to create, edit, and compile your PLC (TwinCAT 3) projects
- Always installed on a Windows machine

XAR:

- TwinCAT real-time kernel which provides the deterministic properties of the PLC SW
- Drivers for industrial fieldbus protocols such as EtherCAT
- Can be installed on both Windows and Beckhoff's own Tc/BSD

Communication protocol:

- Automation Device Specification (ADS)
- Upload new SW
- Read & Write variables (e.g., Visualizer and Measurement)

Introcution to TwinCAT

Consist of two main parts:

- TwinCAT eXtended Automation Engineering (XAE)
- TwinCAT eXtended Automation Runtime (XAR)

Fieldbus interfaces:

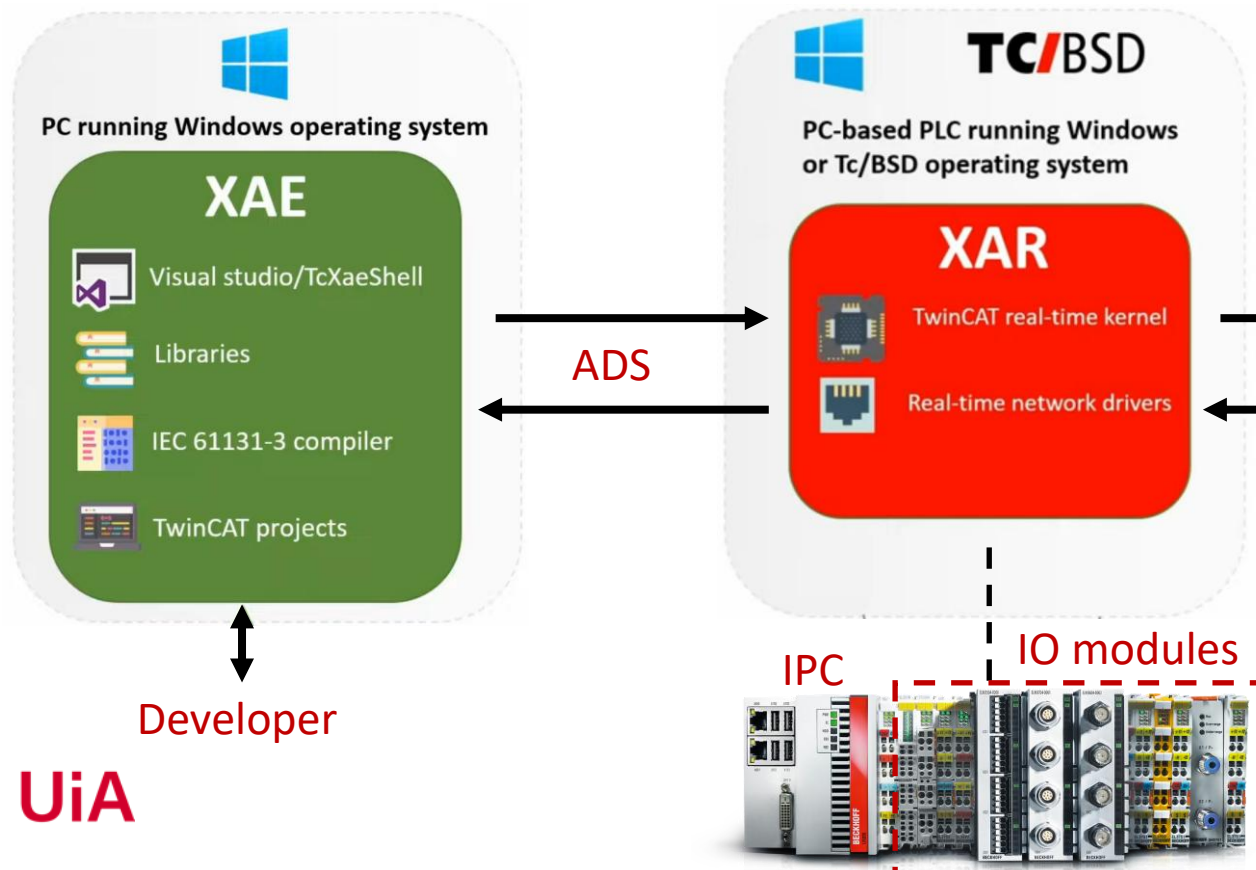
- Communication with various sensors and actuators
- Remote Input/Output (RIO)



Sensors & Actuators, RIO



<https://www.youtube.com/watch?v=0iDn9E0V1iw>



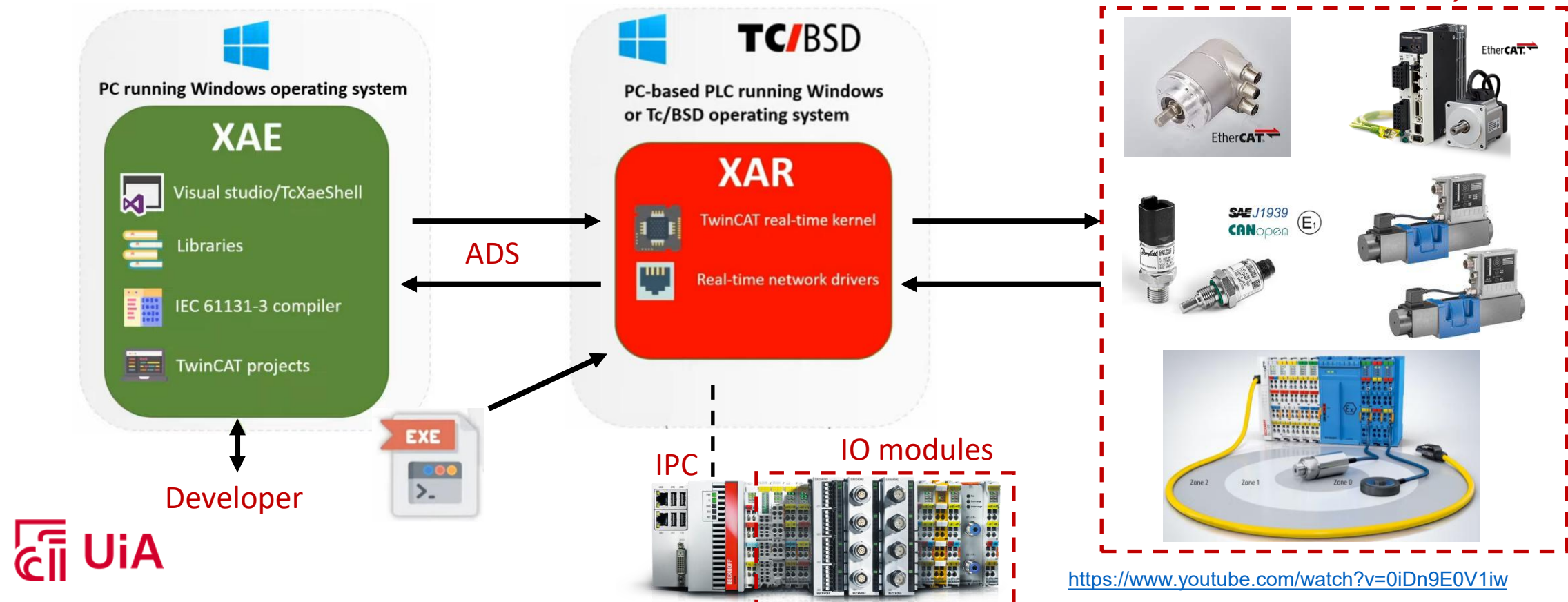
Introcution to TwinCAT

Consist of two main parts:

- TwinCAT eXtended Automation Engineering (XAE)
- TwinCAT eXtended Automation Runtime (XAR)

Compilation:

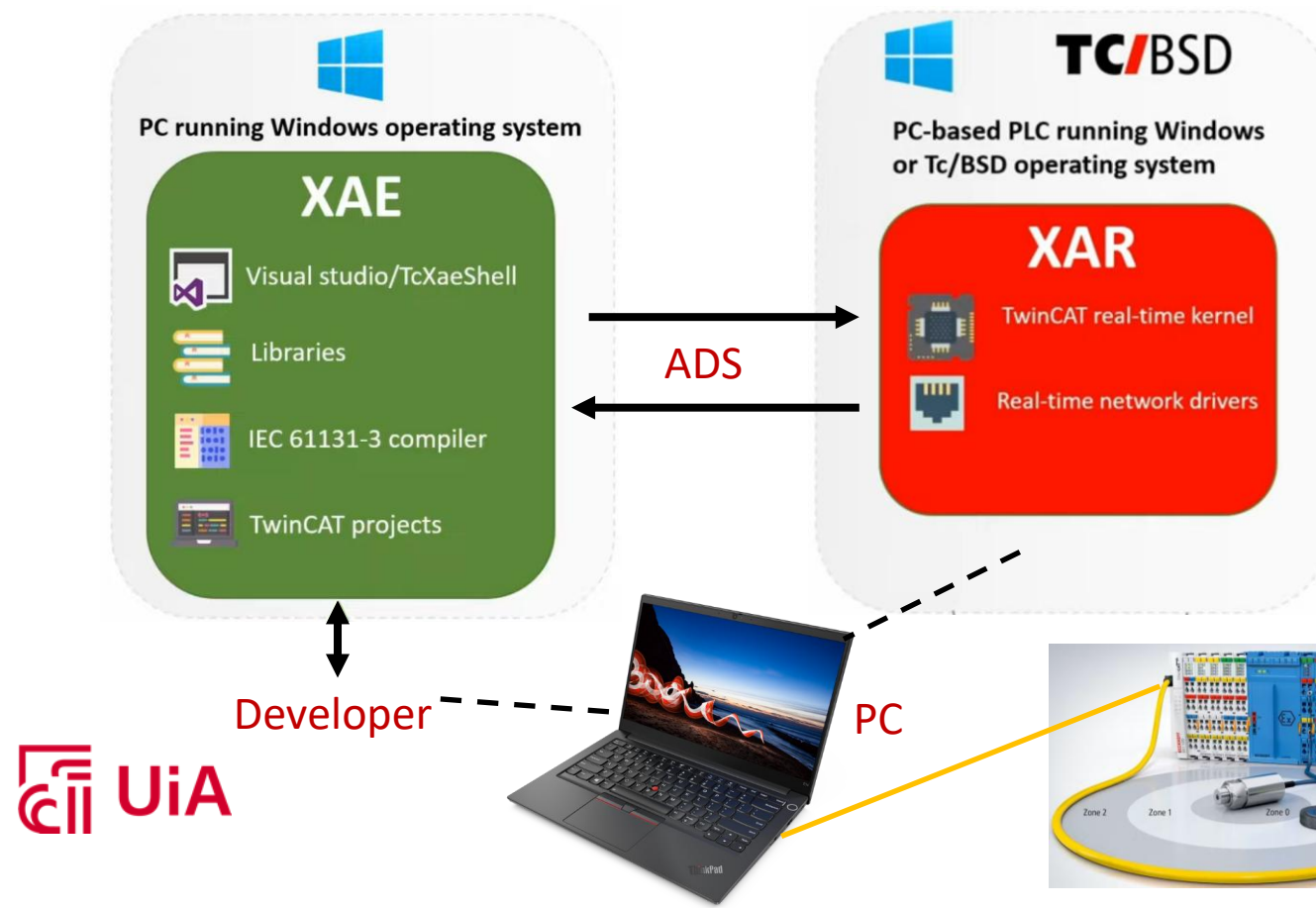
- Executable binary is created, the program can be transferred to the XAR and executed so it is running in the real-time environment of TwinCAT 3 on a PC HW (e.g. Beckhoff IPC)



Introcution to TwinCAT

Consist of two main parts:

- TwinCAT eXtended Automation Engineering (XAE)
- TwinCAT eXtended Automation Runtime (XAR)



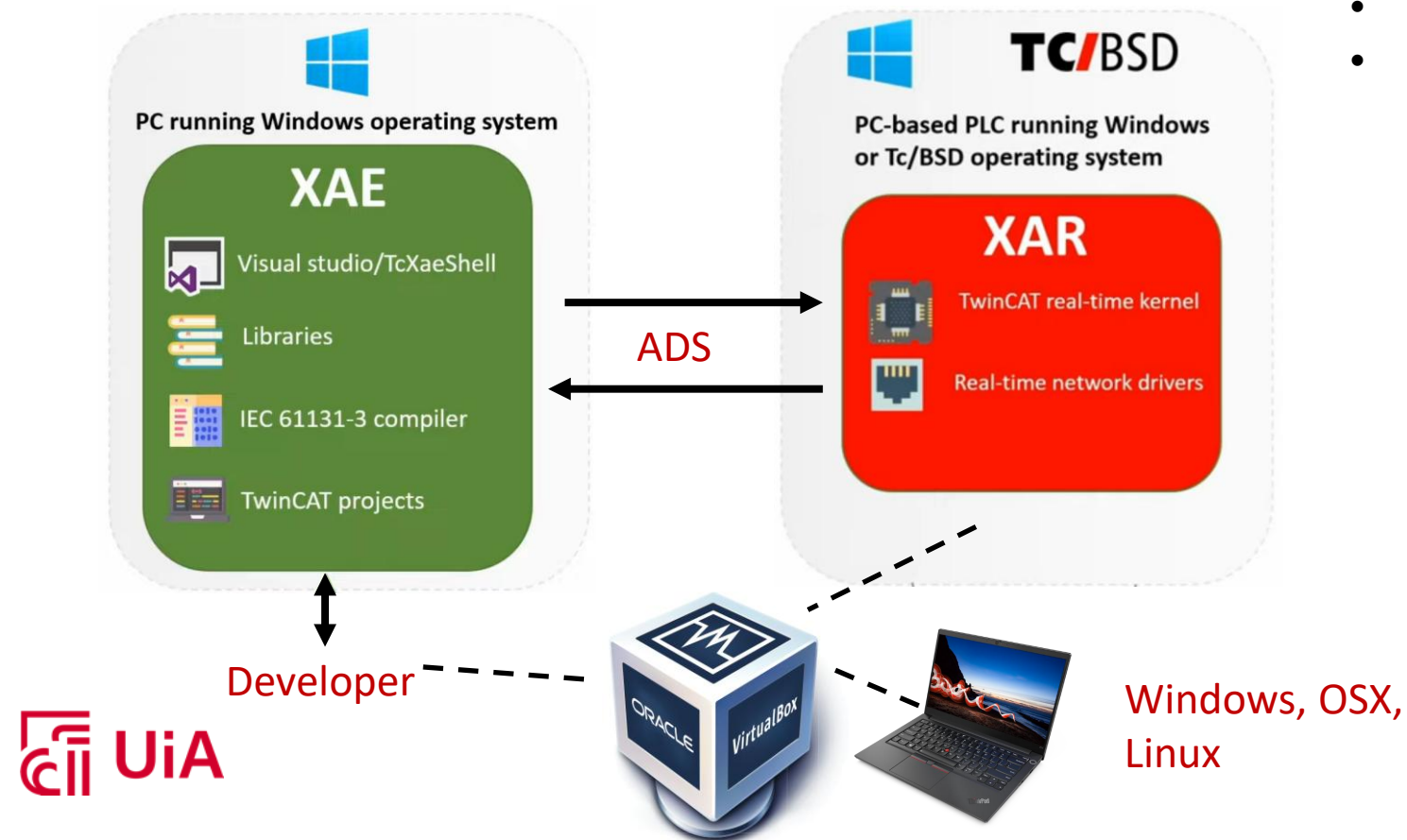
Installation:

- **TwinCAT 4024:**
 - You can choose to install either **XAE** or **XAR**
 - But if you install **XAE** you automatically get **XAR** and can run your TwinCAT software locally on your development PC
- **TwinCAT 4026:**
 - [TwinCAT Package Manager](#)
- You don't need a separate PLC/IPC to try your SW! 😊
- Beckhoff "PLCs" are just ordinary PCs with industrial HW (24v supply etc.) and optimized BIOS-settings for real-time behavior
- You can even connect a RIO to your Ethernet adapter if you have a proper Intel chipset (not gaming PC → Realtek) and control real applications with sufficient real-time behavior
- Finally, you don't need to pay for license for doing software development. But the trial license need to be updated once every week.

Introcution to TwinCAT

Consist of two main parts:

- TwinCAT eXtended Automation Engineering (XAE)
- TwinCAT eXtended Automation Runtime (XAR)



TwinCAT integration:

- In MAS418 we now use **TwinCAT 4026**. You can choose if you want to use your own PC and directly install the **TwinCAT Package Manager** and **TwinCAT Standard** (you need both **Engineering** and **Runtime**) or use the lab PC's. PS: most lab PC's misses the Runtime.
- You can also use virtual machine (VirtualBox).
- In part 3 you will work in groups using real Beckhoff IPSc ([CX2033](#)) running TC/BSD



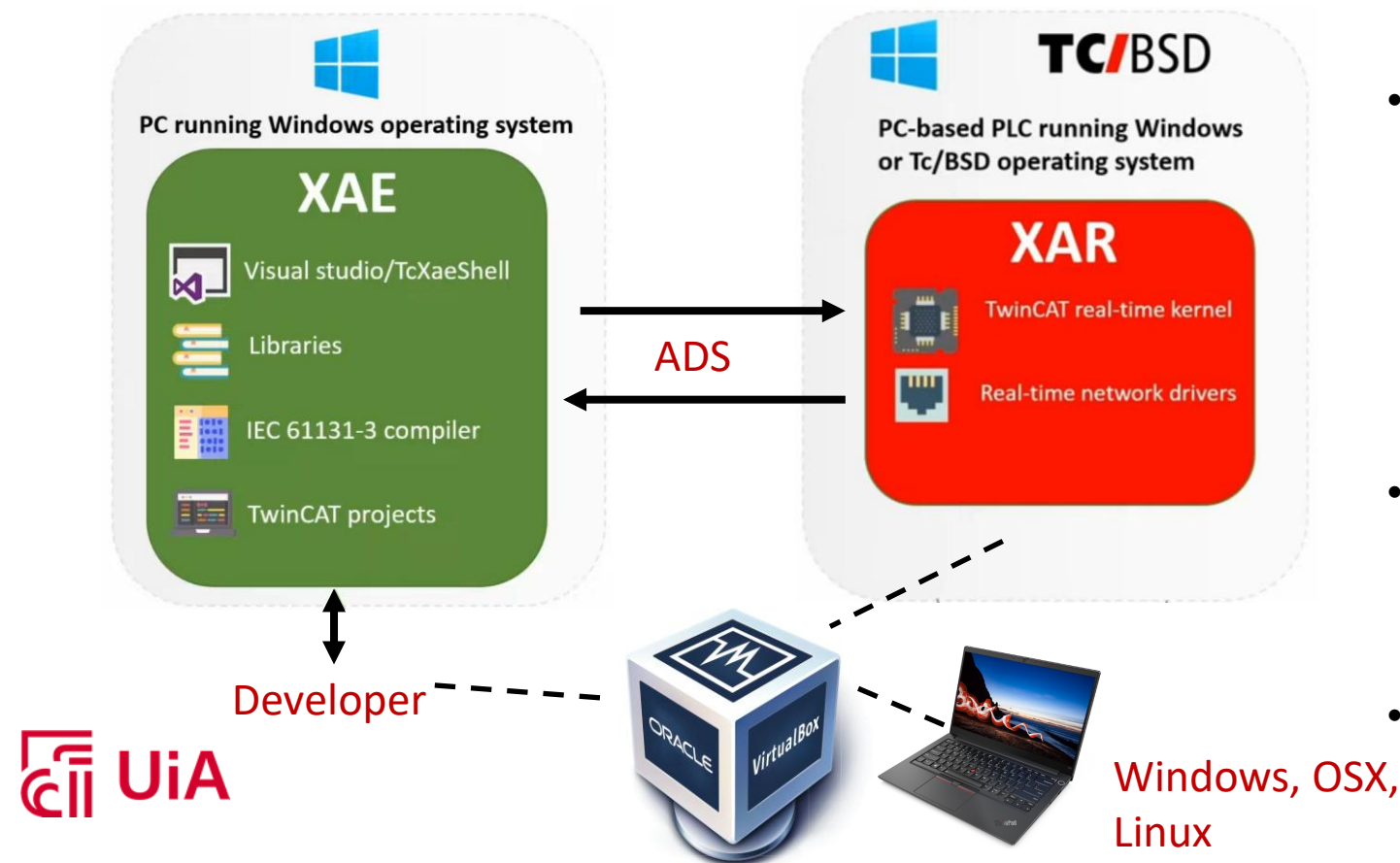
Technical data	CX2033
Processor	AMD Ryzen™ V1202B 2.3 GHz
Number of cores	2
Flash memory	slot for CFast card
Main memory	8 GB DDR4 RAM (expandable ex factory to 16 GB)

<https://www.youtube.com/watch?v=0iDn9E0V1iw>

Introcution to TwinCAT

Consist of two main parts:

- TwinCAT eXtended Automation Engineering (XAE)
- TwinCAT eXtended Automation Runtime (XAR)



XAE integration:

- When you install TwinCAT 3 **XAE** you get the TwinCAT XAE Shell development environment, which is based on Visual Studio (**VS**)



TwinCAT XAE Shell (TcXaeShell)

- Based on Visual Studio
- Included in TwinCAT 3 XAE

- The TcXaeShell is basically a version of **VS** where the possibility for all other language such as C#/C++ have been removed, but all the other functionality of VS is left intact
 - Build command, search and replace function, integrated version control with Git, etc.
- However, if you are already using VS (2013, 2015, 2017, 2019, 2022) you can integrate the **XAE** installation with **VS**. But I will recommend all to use the Shell in this course.
- In **TwinCAT 2026** the Shell is now based on VS 2022 64-bit

<https://www.youtube.com/watch?v=0iDn9E0V1iw>

Difference to SW running in OS

- Difference between traditional “IT” software running as a process in an operating system compared to writing software that runs in TwinCAT 3

IT software

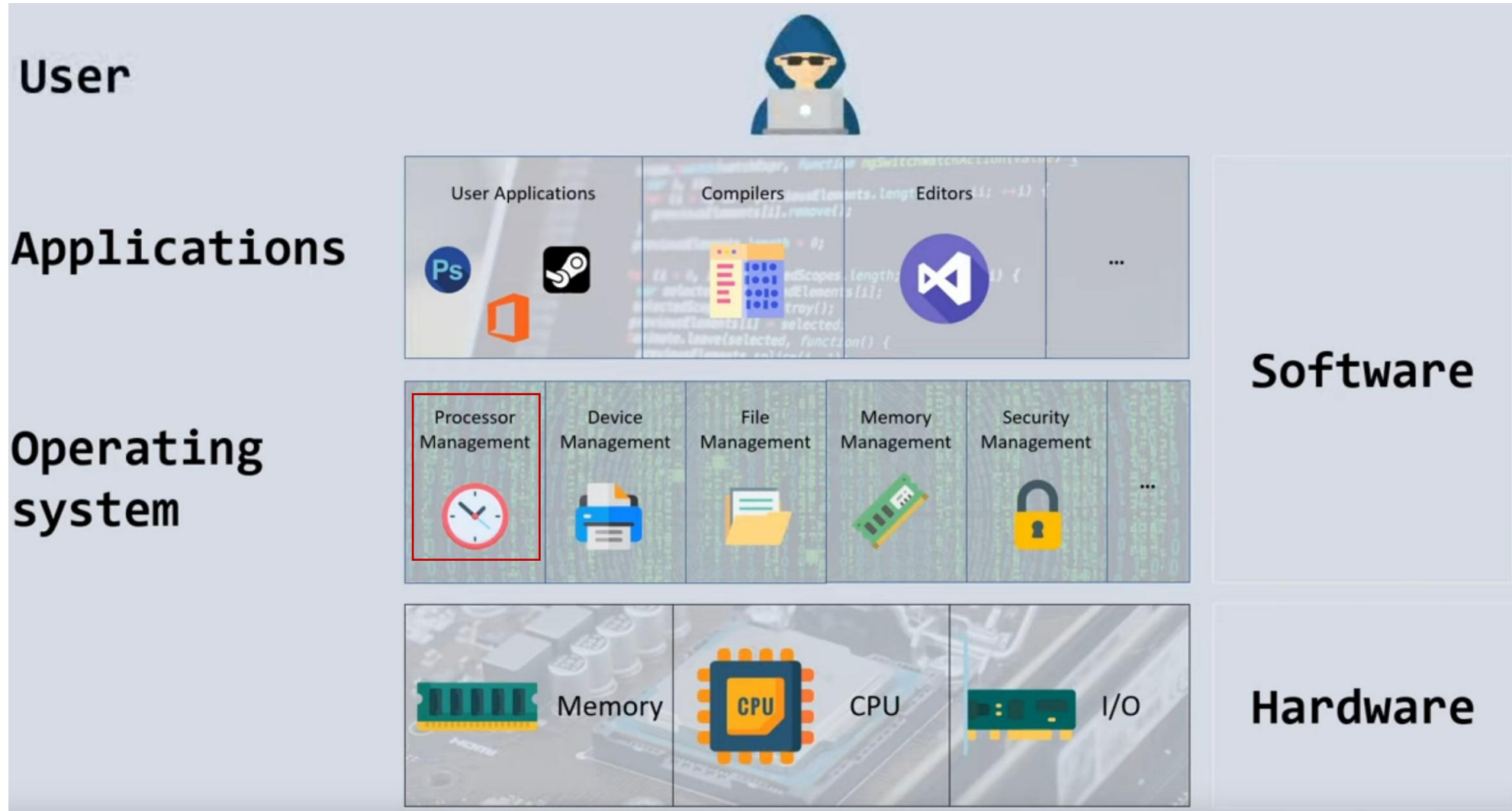


Automation software



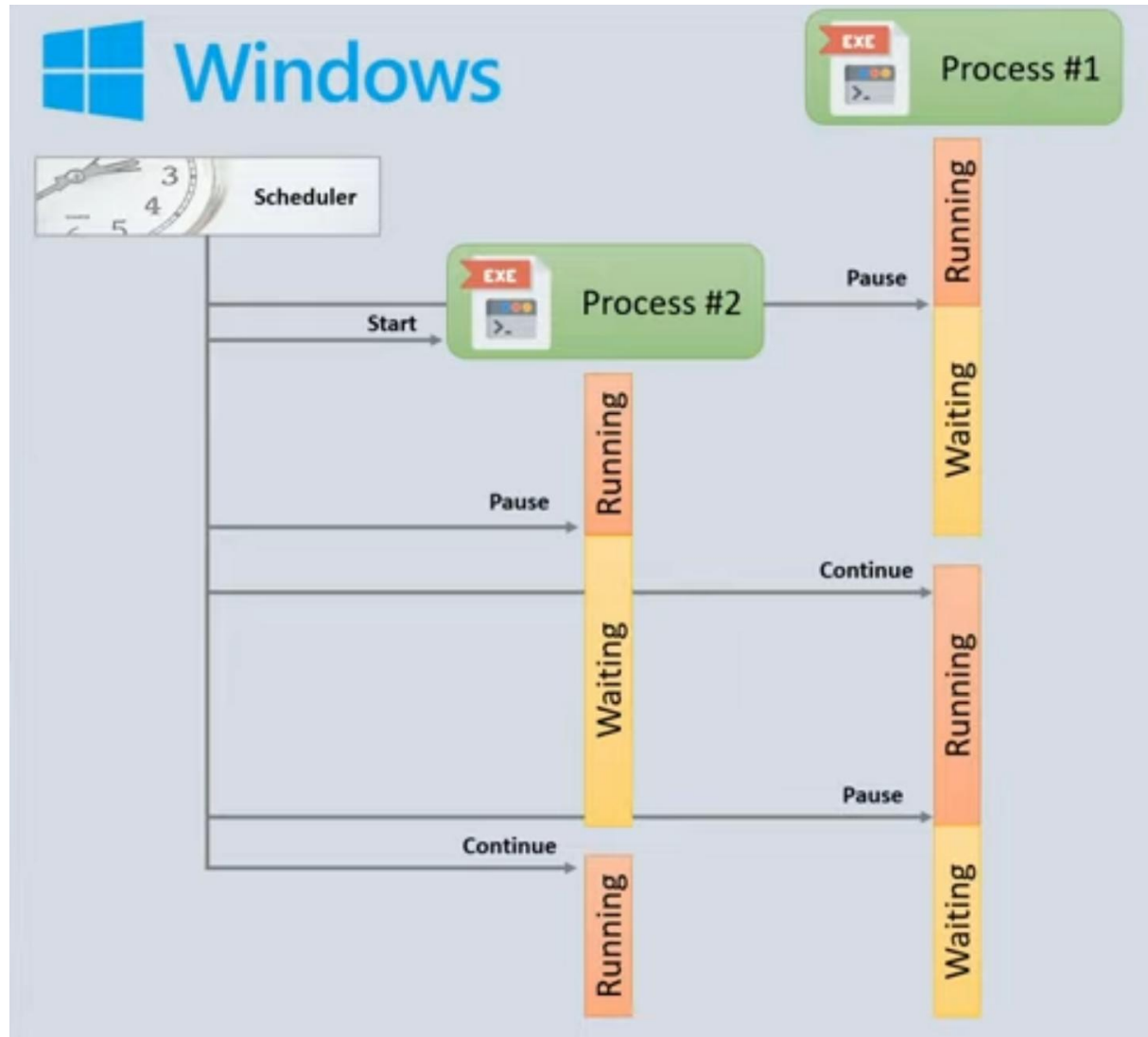
Difference to SW running in OS

- Basics



Difference to SW running in OS

- Processor management (Windows)



Scheduling policies

- First in, first out (FIFO)
- Shortest job first
- Round-robin
- Fixed priority pre-emptive scheduling
- Manual scheduling
- Priority scheduling
- ...

Windows 10

- Multilevel feedback queue

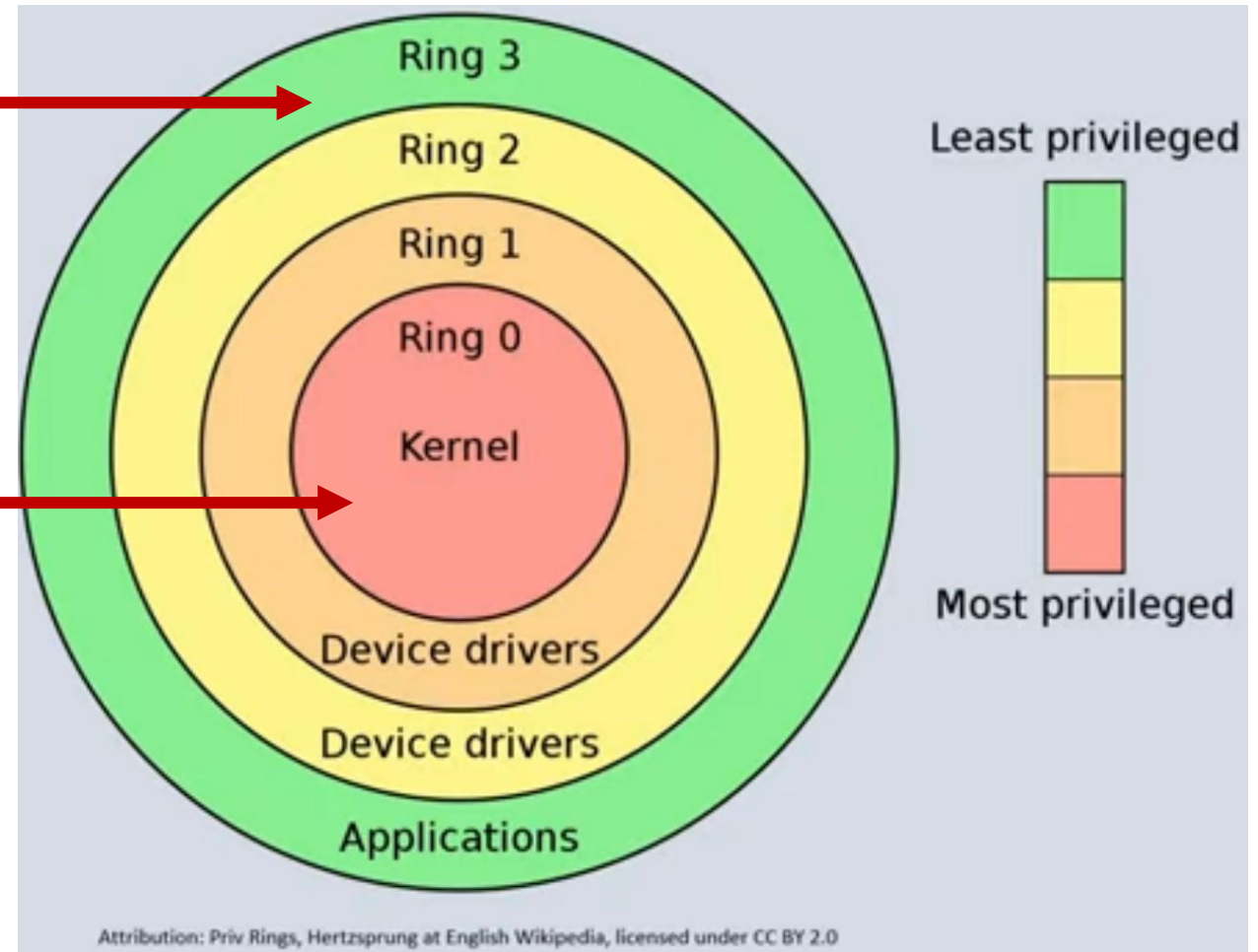
Difference to SW running in OS

Windows applications

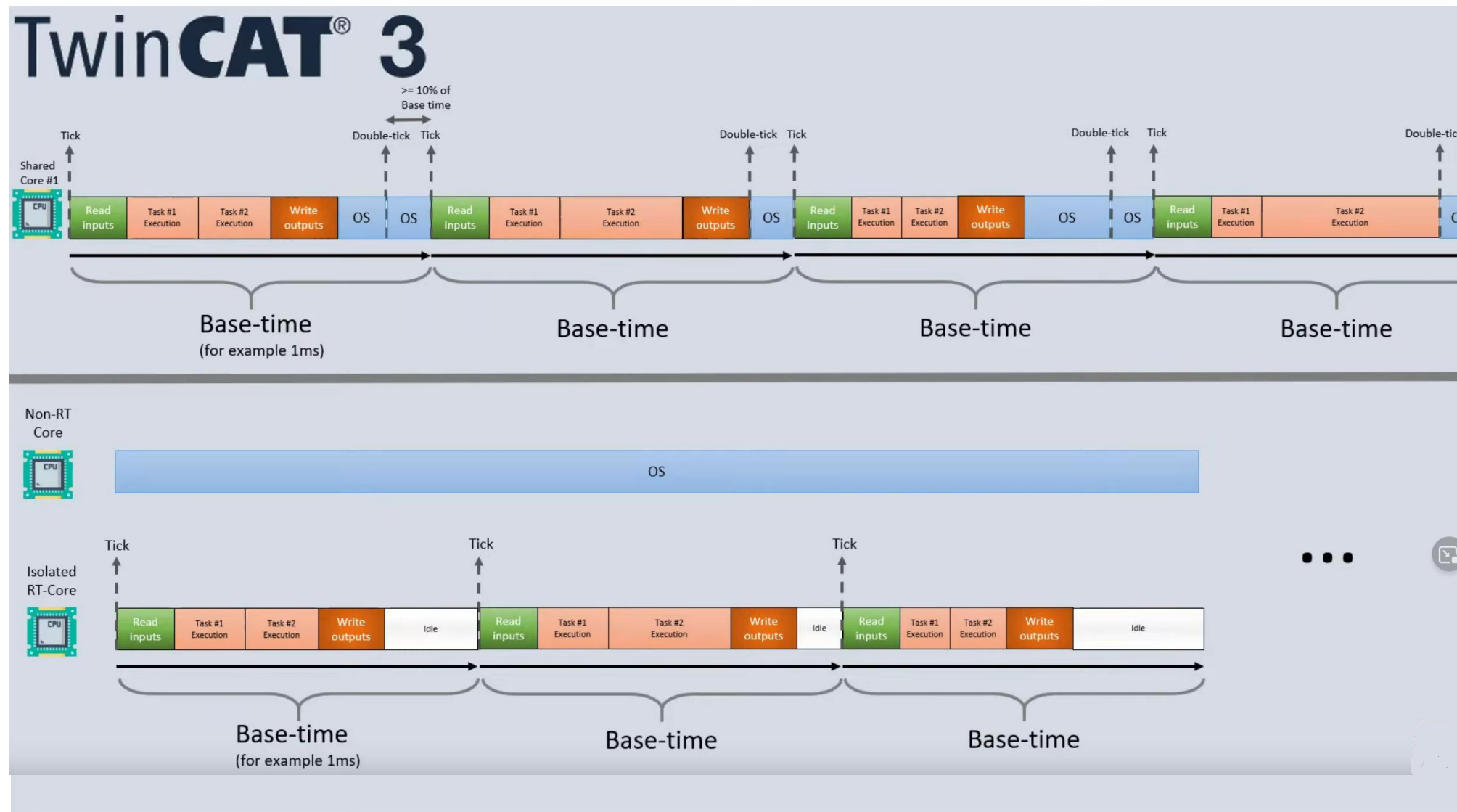
Windows networking
Applications (processes)
TwinCAT HMI
...

Windows kernel & TwinCAT runs here

TwinCAT scheduler
I/O, fieldbuses (EtherCAT...)
Motion Control
...



Difference to SW running in OS

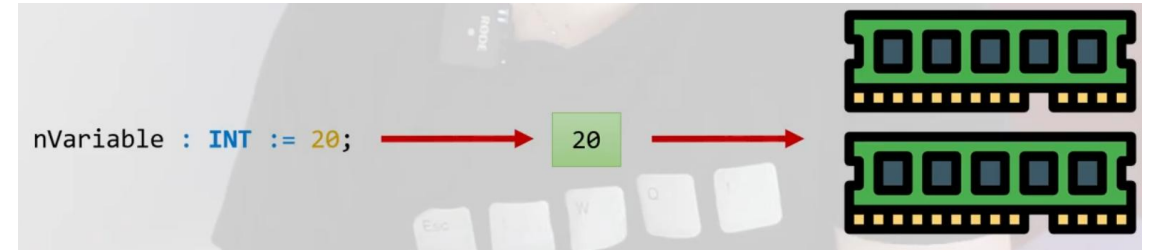


Part III: Data types & Std. library

1. Basic data types
2. Time
3. Pointers
4. Reference
5. Arrays
6. Enumeration
7. Standard library

Basic data types

- **What is a variable?**
 - **Variables** are used to store information to be references and manipulated in a computer program
 - They provide a way of labeling **data** with a descriptive name, so our programs can be understood more clearly by the developers
- Think of **variables** as container that hold information
 - Their purpose is to label and store **data** in memory
 - This **data** can then be used throughout your program
- When creating a **variable**, you need to define what **data** it should have from the **IEC61131-3** standard



All variables are declared between:

```
1 PROGRAM TestProgram
2
3 VAR
4     bTestVariable : BOOL;
5     fTestVariable : REAL;
6     nTestVariable : INT;
7 END_VAR
```

Giving good names to variables is a task that should not be underestimated - see:

<https://alltwincat.com/2019/02/11/plc-naming-conventions/>

Basic data types

- Booleans

TYPE	VALUES	MEMORY CONSUMPTION	EXAMPLE
BOOL	FALSE (0) TRUE (1)	8 bits (1 byte)	1 bJustAVariable 2 : BOOL 3 := TRUE;

1. Name

2. Datatype

3. Optional assigned (initial) value

- If not assigned, the default value for a boolean is **FALSE**

Basic data types

• Integer

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
INT	-32768	32767	16 bits (2 bytes)	<code>nThisIsANumber : INT := -4232;</code>
UINT	0	65535	16 bits (2 bytes)	<code>nThisIsANumber : UINT := 44000;</code>
SINT	-128	127	8 bits (1 byte)	<code>nThisIsANumber : SINT := -3;</code>
USINT	0	255	8 bits (1 byte)	<code>nThisIsANumber : USINT := 241;</code>
DINT	-2147483648	2147483647	32 bits (4 bytes)	<code>nThisIsANumber : DINT := 1362992311;</code>
UDINT	0	4294967295	32 bits (4 bytes)	<code>nThisIsANumber : UDINT := 3453331141;</code>
LINT	-9223372036854775808	9223372036854775807	64 bits (8 bytes)	<code>nThisIsANumber : LINT := -223372036854775807;</code>
ULINT	0	18446744073709551615	64 bits (8 bytes)	<code>nThisIsANumber : ULINT := 13246311073709551615;</code>

Basic data types

• Integer

Base Converter:

<https://www.rapidtables.com/convert/number/base-converter.html>

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
BYTE	0	255	8 bits (1 byte)	nSomeVariable : BYTE := 2#1011_0011;
WORD	0	65535	16 bits (2 bytes)	nSomeVariable : WORD := 16#02AE;
DWORD	0	4294967295	32 bits (4 bytes)	nSomeVariable : DWORD := 16#02AE_FFFF;
LWORD	0	18446744073709551615	64 bits (8 bytes)	nSomeVariable : LWORD := 10#18446744073709551615;

```
nSomeVariable : BYTE := 2#1011_0011;
```

```
nSomeVariable.3;
```

Basic data types

- Floating point

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
REAL	-3.4×10^{38}	3.4×10^{38}	32 bits (4 bytes)	<code>fPi : REAL := 3.14159265359;</code>
LREAL	$-1.7976931348623158^{308}$	1.7976931348623158^{308}	64 bits (8 bytes)	



```
fThisIsAFloat : LREAL := 9814123441244124121244132432342342343115312341.331112571442123;
```


Basic data types

- String

TYPE	MEMORY CONSUMPTION	EXAMPLE	
STRING	80 (+1) bytes	<code>sHelloWorldString : STRING := 'Hello World!';</code>	ASCII-coded
STRING(300)	300 (+1) bytes	<code>sThisIsAStringVariable : STRING(300) := 'This is a string';</code>	ASCII-coded

Basic data types

ASCII-character Table

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Basic data types

• String

TYPE	MEMORY CONSUMPTION	EXAMPLE	
STRING	80 (+1) bytes	<code>sHelloWorldString : STRING := 'Hello World!';</code>	ASCII-coded
STRING(300)	300 (+1) bytes	<code>sThisIsAStringVariable : STRING(300) := "This is a string";</code>	ASCII-coded
WSTRING	80 (+1) words 160 (+2) bytes	<code>wsHelloWorld : WSTRING := "Hello World!";</code>	UNICODE-coded
WSTRING(300)	300 (+1) words 600 (+2) bytes	<code>wsJustAVariable : WSTRING(300) := "GHäöæå";</code>	UNICODE-coded

Å Æ Ø

Basic data types - Type conversion

USINT



1 byte

USINT
UPPER BOUND

255

UINT



2 bytes

UINT
UPPER BOUND

65535

VAR

nOne Usint : USINT;

nOne_Uint : UINT := 259;

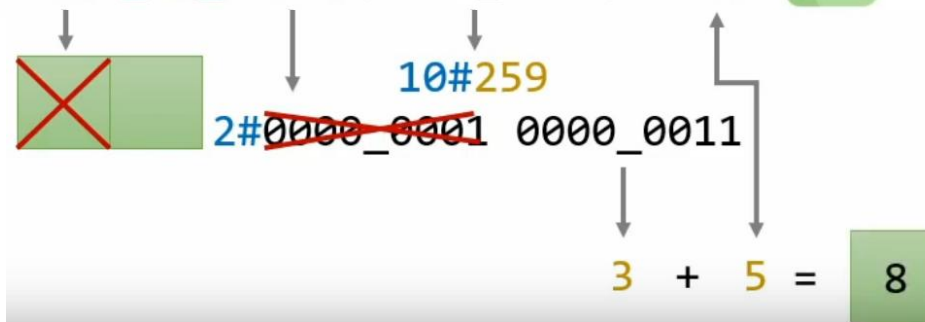
END_VAR

nOne_Uint := nOne_Usint + 5; ✓

nOne_Usint := nOne_Uint + 5; ✗

✗ Cannot convert type 'UINT' to type 'USINT'

nOne_Usint := UINT_TO_USINT(nOne_Uint) + 5; ✓



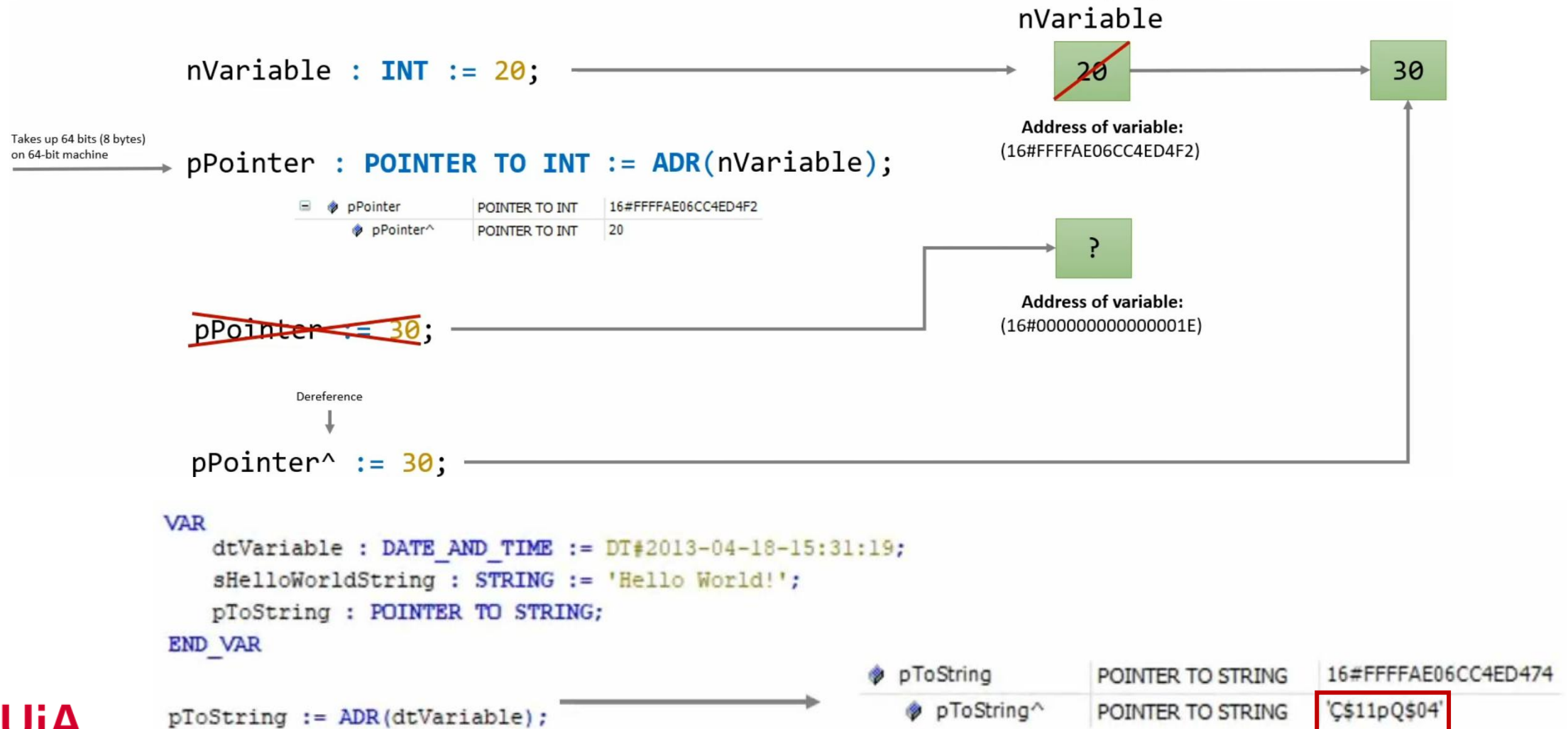
Basic data types

- There are a few other **data types** available in the **IEC standard** and TwinCAT3, but these together with the ones in the next lecture are the most common ones
 - **Se Beckhoff documentation:**
https://infosys.beckhoff.com/english.php?content=../%2Fcontent%2F1033%2Ftc3_plc_intro%2F2529388939.html&id
- In TwinCAT3, and in PLCs in general, all type **declaration** is done at **compile time**
 - You have to **declare** the **data types** of your **variable** before you use them, and the type cannot be changed during the execution of the program
 - Implicitly trying to convert from one type to another will generally result in a **compiler warning** or **error**
 - In PLCs you generally **allocate** memory upfront
 - Dynamic **memory allocation** the way it can be done in for example C++ can be done in TwinCAT, although it is generally avoided

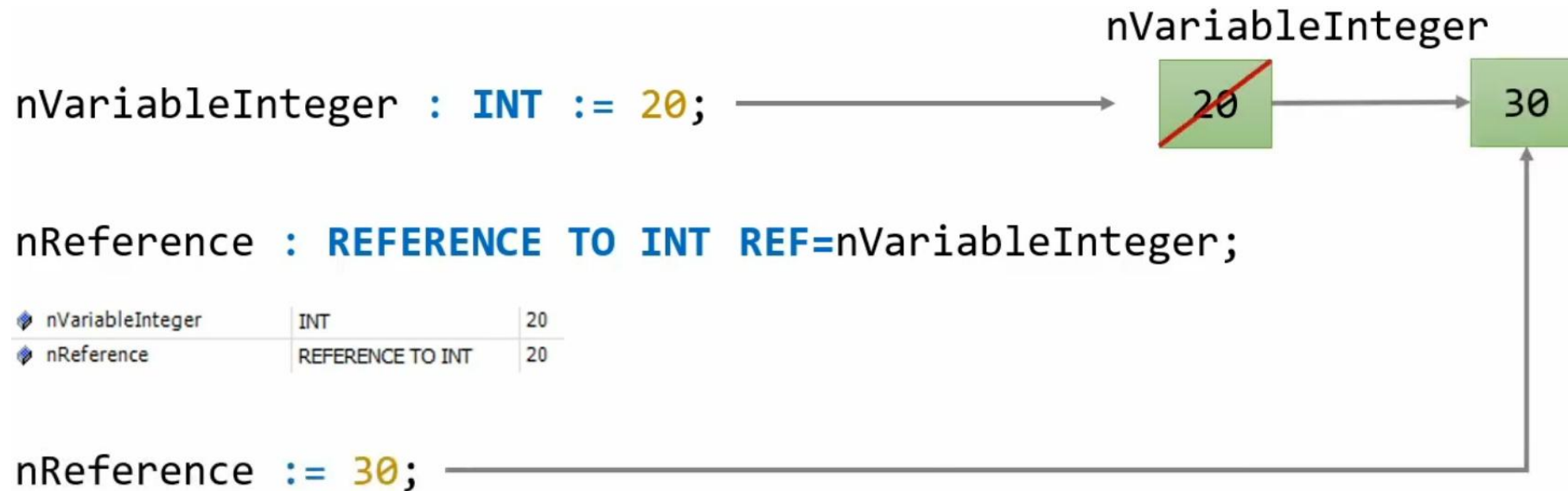
Time

TYPE	LOWER BOUND	UPPER BOUND	MEMORY CONSUMPTION	EXAMPLE
TIME	0 [ms]	4294967295 [ms]	32 bits (4 bytes)	tVariable : TIME := T#1D4H5M2S132MS;
TIME_OF_DAY (TOD)	0 (00:00:00:000) [ms]	4294967295 (23:59:59:999) [ms]	32 bits (4 bytes)	tdVariable : TIME_OF_DAY := TOD#11:14:45.577;
DATE	0 (01.01.1970) [s]	4294967295 (7 February 2106) [s]	32 bits (4 bytes)	dDateVariable : DATE := D#1982-03-01;
DATE_AND_TIME (DT)	0 (01.01.1970, 00:00) [s]	4294967295 (7 February 2106, 6:28:15) [s]	32 bits (4 bytes)	dtVariable : DATE_AND_TIME := DT#2013-04-18-15:31:19;
LTIME	0 [ns]	213503d23h34m33s709ms551us615ns [ns]	64 bits (8 bytes)	ltVar : LTIME := LTIME#213503D23H34M33S709MS551US615NS;

Pointers



Reference



- Easier to use
- Type safe

```
dtVariable : DATE_AND_TIME := DT#2013-04-18-15:31:19;
nVariableInteger : INT := 122;
nReference : REFERENCE TO INT REF= dtVariable;
```

Cannot convert type 'DATE_AND_TIME' to type 'REFERENCE TO INT'

Reference

- For more details about pointers and references, and how they differ to in C++, see the link below
- In general:
 - use **reference** when you **can**
 - and **pointers** when you **have to**

Arrays

```
aOneArray : ARRAY[1..5] OF INT := [10, 20, 30, 40, 50];
```

```
aOneArray[2] := 133;
```



```
aOneArray : ARRAY[0..2] OF LREAL;
```

```
aOneArray : ARRAY[-1..1] OF STRING;
```

Arrays

Alternative #1

```
a2dArray : ARRAY[1..2, 1..3] OF REAL;
```

a2dArray		
2dArray[1,1]	2dArray[1,2]	2dArray[1,3]
2dArray[2,1]	2dArray[2,2]	2dArray[2,3]

Alternative #2

```
a2dArray2 : ARRAY[1..2] OF ARRAY[1..3] REAL;
```

a2dArray2		
2dArray[1][1]	2dArray[1][2]	2dArray[1][3]
2dArray[2][1]	2dArray[2][2]	2dArray[2][3]

Arrays

[-] a2dArray	ARRAY [1..2, 1..3] OF REAL
a2dArray[1, 1]	REAL
a2dArray[1, 2]	REAL
a2dArray[1, 3]	REAL
a2dArray[2, 1]	REAL
a2dArray[2, 2]	REAL
a2dArray[2, 3]	REAL
[-] a2dArray2	ARRAY [1..2] OF ARRAY [1..3] OF REAL
[-] a2dArray2[1]	ARRAY [1..3] OF REAL
a2dArray2[1][1]	REAL
a2dArray2[1][2]	REAL
a2dArray2[1][3]	REAL
[-] a2dArray2[2]	ARRAY [1..3] OF REAL
a2dArray2[2][1]	REAL
a2dArray2[2][2]	REAL
a2dArray2[2][3]	REAL

Arrays



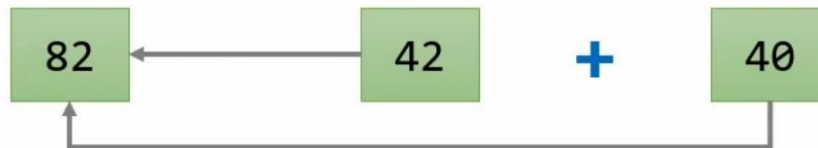
VAR

```
aOneArray : ARRAY[1..5] OF INT := [10, 20, 30, 40, 50];  
nOneInt : INT;
```

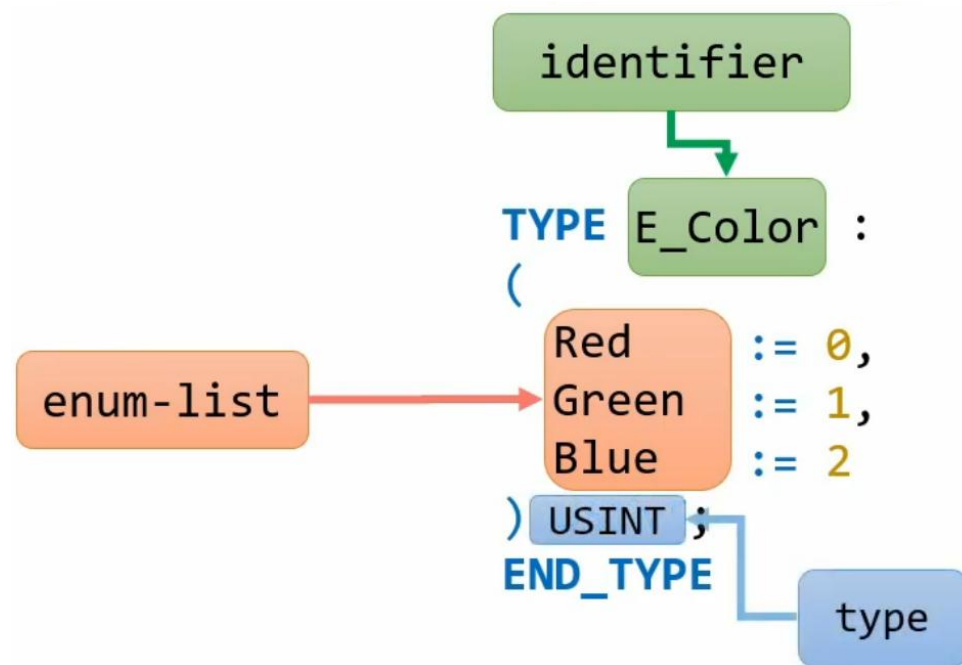
END_VAR

```
aOneArray[1] := 42;
```

```
nOneInt := aOneArray[1] + aOneArray[4];
```



Enumeration



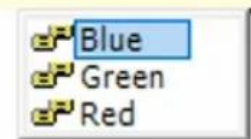
VAR

```
eLightTower : E_Color;
```

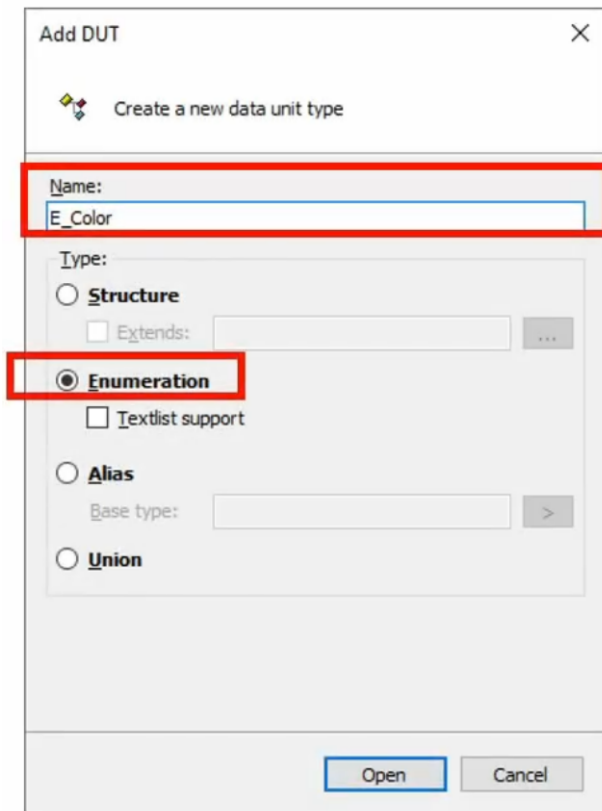
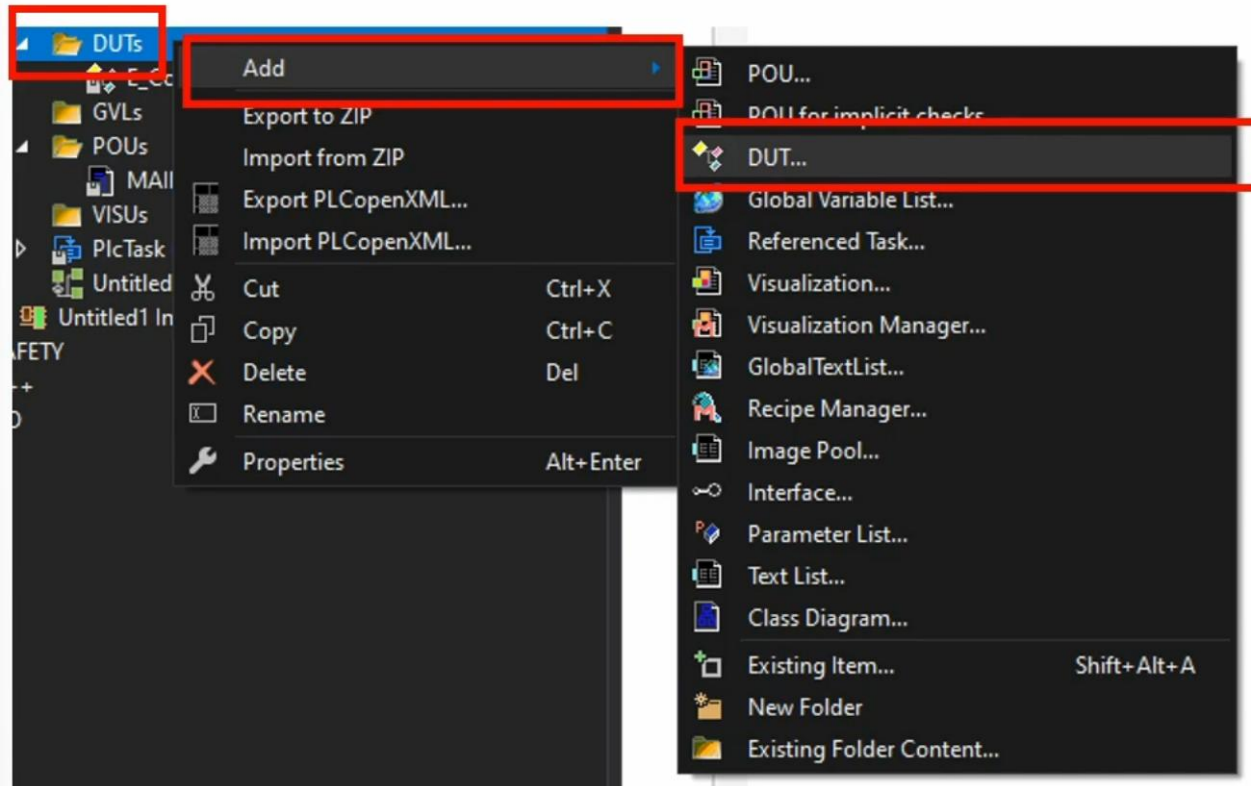
END_VAR

```
eLightTower := E_Color.Red;
```

```
eLightTower := E_Color.
```



Enumeration



```
E_Color
1 {attribute 'qualified_only'}
2 {attribute 'strict'}
3 TYPE E_Color :
4 (
5     enum_member := 0
6 );
7 END_TYPE
```

Standard library

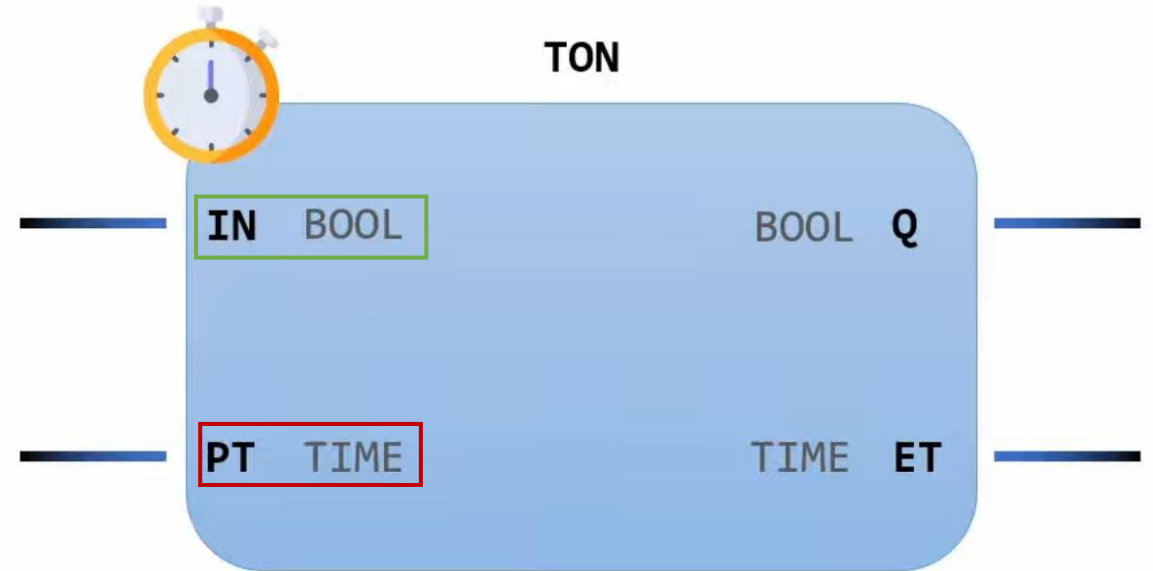
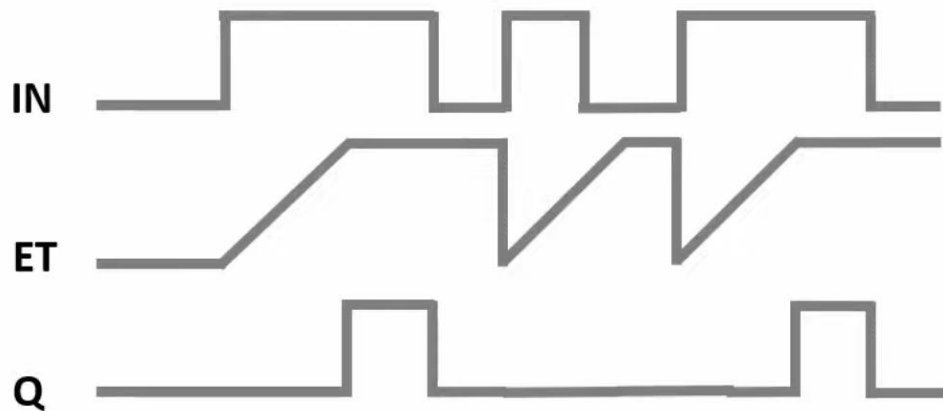
Some basic **PLC functions** and **function blocks** from the **TwinCAT** (Tc2_ Standard) library:

- **Timer ON delay – TON**
- **Timer OFF delay – TOF**
- **Rising edge detection – R_TRIG**
- **Falling edge detection – F_TRIG**
- **Concatenate two STRINGs – CONCAT**
 - One of many string functions

Standard library

• TON

- Turns an output **ON** after a delay
- The **input PT** is the time of the delay (wait period)
- The **input IN** is used to reset the timer with a falling edge.
- The **output ET** is the elapsed time that displays the amount of time since the start of the timer
- The **output Q** is set to true after the timer has elapsed **ET** amount of time



```

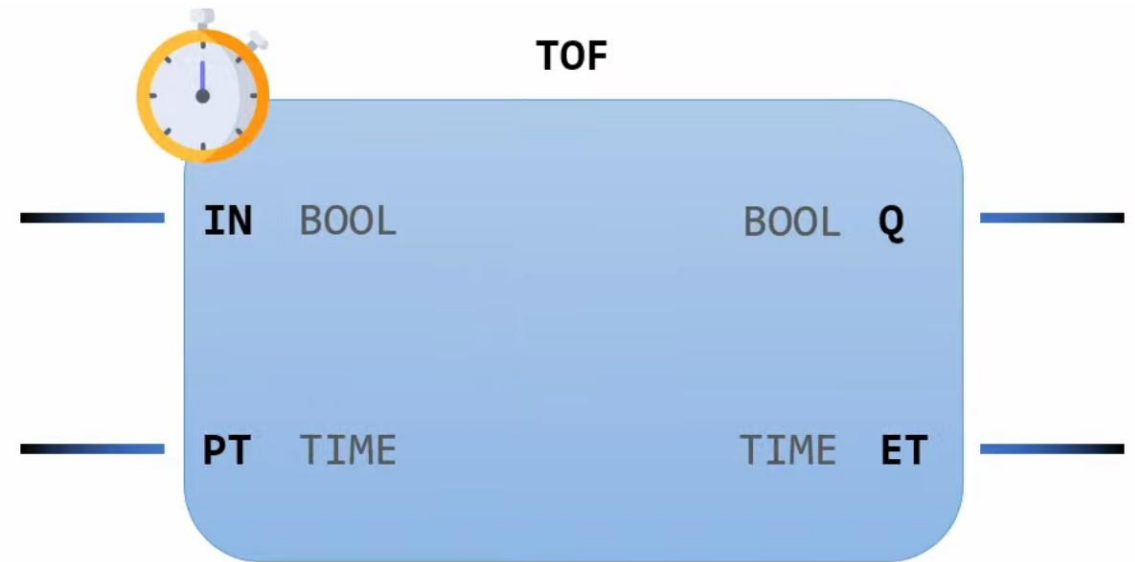
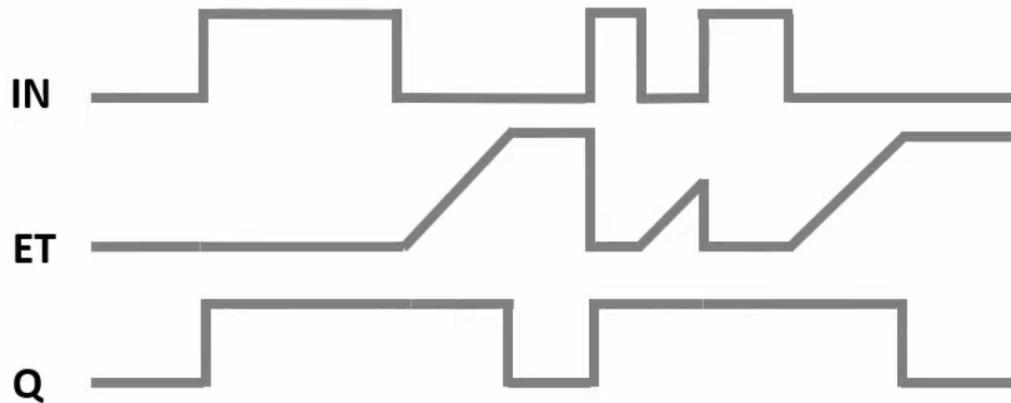
VAR
    fbTon : TON := (PT := T#3S);
    bBool : BOOL;
END_VAR

fbTon(IN := bBool);
  
```

Standard library

• TOF

- The **inverse** of **TON**
- Instead of having the timer elapsing when **IN** is high, it is counted when **IN** is low
 - i.e., when a falling edge is a trigger for the **FB**
- Another difference is that once the timer has elapsed the **output Q** goes low as opposed to high in **TON**



```

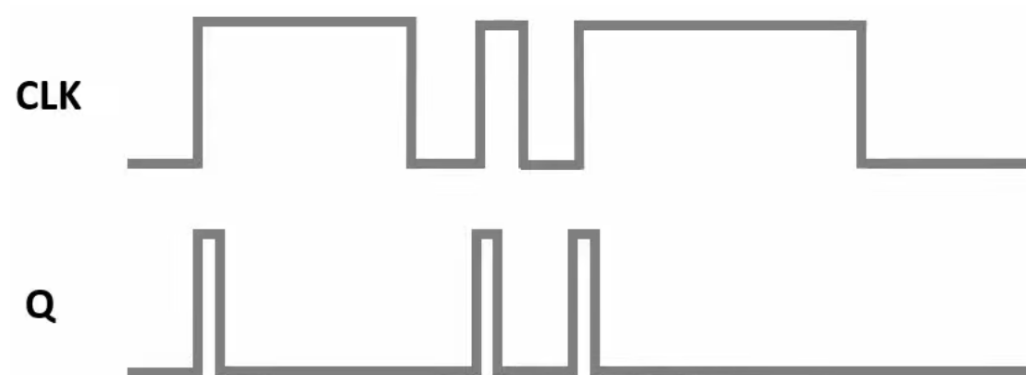
VAR
  fbTof : TOF := (PT := T#3S);
  bBool : BOOL;
END_VAR

fbTof(IN := bBool);
  
```


Standard library

• R_TRIG

- Used to **detect** a **rising edge**
- The **input CLK** is the signal to detect
- The **output Q** is the edge detected
- Each time the **FB** is called, **Q** will return a false until **CLK** has been false followed by a **rising edge**
 - i.e., the value true



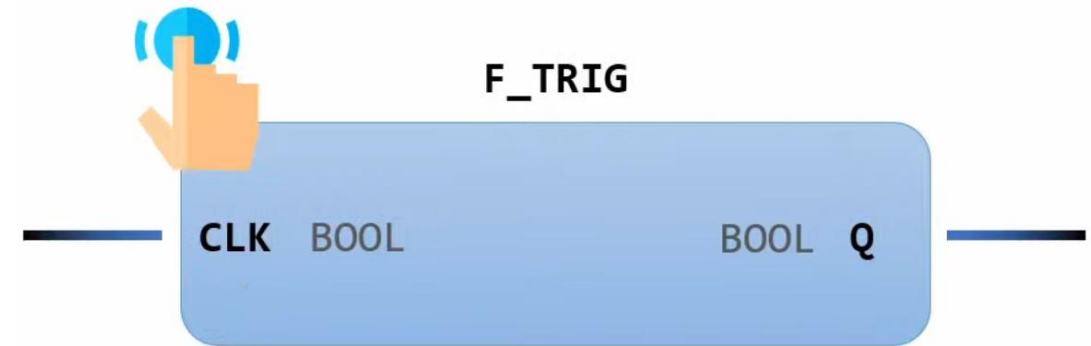
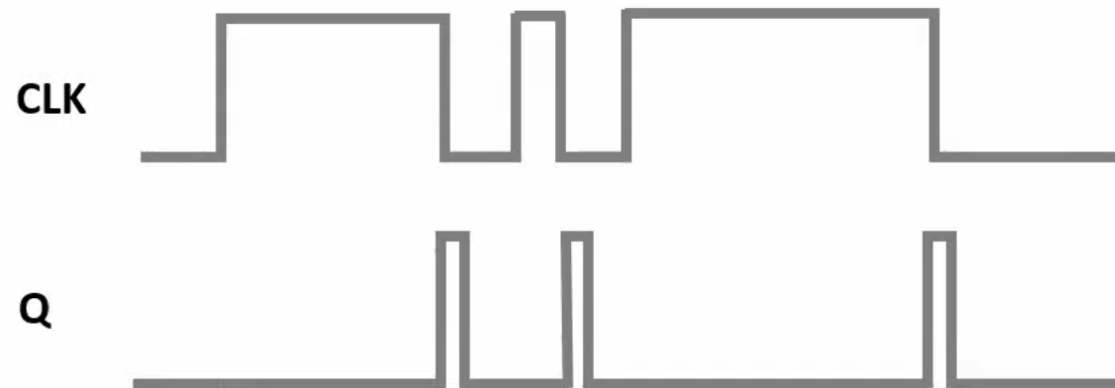
```
VAR
    fbRTrig : R_TRIG;
    bBoolean : BOOL;
END_VAR

fbRTrig(CLK := bBoolean);
```

Standard library

• F_TRIG

- Works as **R_TRIG** but **detects** a **falling edge** instead of a **rising edge**
- Each time the **FB** is called **Q** will return false until **CLK** has been true, followed by a **falling edge**
 - i.e., the value false



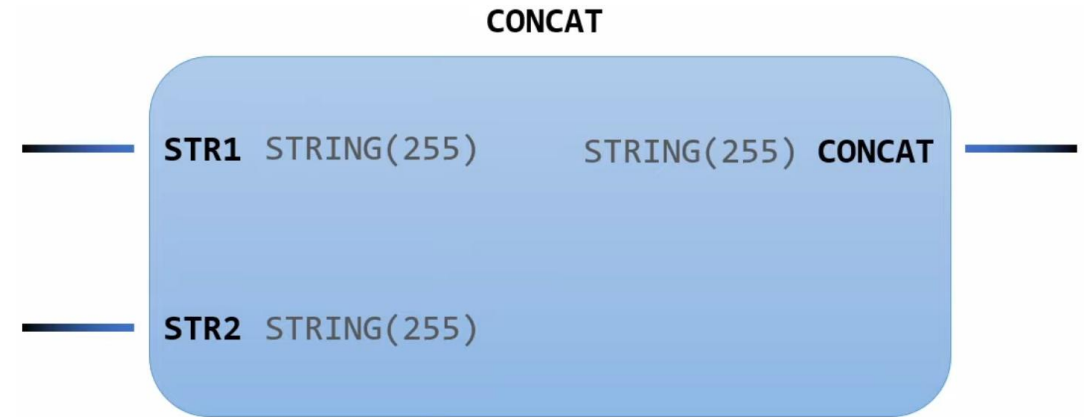
```
VAR
    fbFTrig : F_TRIG;
    bBoolean : BOOL;
END_VAR

fbFTrig(CLK := bBoolean);
```

Standard library

• CONCAT

- Concatenates **two STRINGS into one**
- **Strings** can be longer than 255 characters
- The **CONCAT** function only works with strings up to 255 characters (pr input) and the resulting SRING is 255
- There is a **FB** called **CONCAT2** that can take **STRINGS** of an arbitrary length and concatenate them



VAR

```
sString1 : STRING(255) := 'Hello ';
```

```
sString2 : STRING(255) := 'World!';
```

```
sResultString : STRING(255);
```

END_VAR

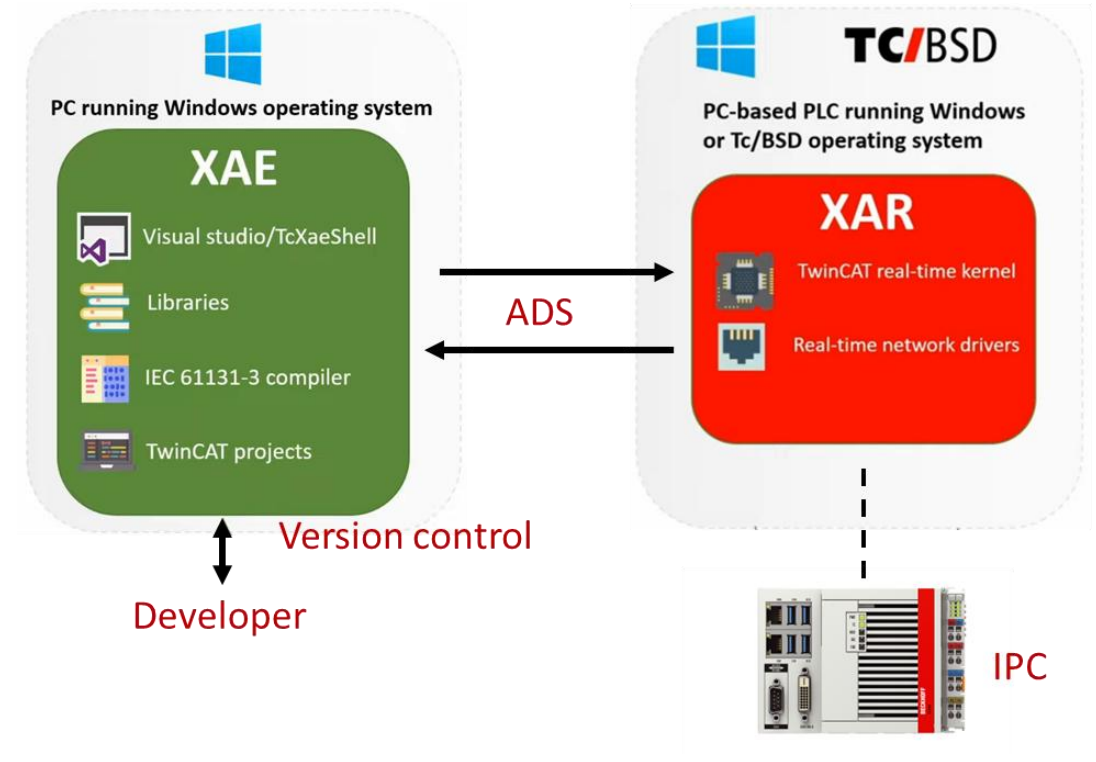
```
sResultString := CONCAT(STR1 := sString1, STR2 := sString2);
```

└─> 'Hello World!'

Summary

Summary

- **Course Introduction**
- **Introduction to PLC programming**
 - History
 - IEC61131-3 standard
 - Version-control
- **TwinCAT basics**
 - Introduction to TwinCAT
 - Difference to SW running in OS
- **Data types & Std. library**
 - Basic data types
 - Time
 - Pointers
 - Reference
 - Arrays
 - Enumeration
 - Standard library



Next Lecture

Procedural-oriented PLC programming:

- I. Structures & functions
- II. Instructions
- III. Crane application and simulation in TwinCAT

Juli 2025								August 2025								September 2025							
Uke	Ma	Ti	On	To	Fr	Lø	Sø	Uke	Ma	Ti	On	To	Fr	Lø	Sø	Uke	Ma	Ti	On	To	Fr	Lø	Sø
27		1	2	3	4	5	6	31					1	2	3	36	1	2	3	4	5	6	7
28	7	8	9	10	11	12	13	32	4	5	6	7	8	9	10	37	8	9	10	11	12	13	14
29	14	15	16	17	18	19	20	33	11	12	13	14	15	16	17	38	15	16	17	18	19	20	21
30	21	22	23	24	25	26	27	34	18	19	20	21	22	23	24	39	22	23	24	25	26	27	28
31	28	29	30	31				35	25	26	27	28	29	30	31	40	29	30					

Oktober 2025								November 2025								Desember 2025							
Uke	Ma	Ti	On	To	Fr	Lø	Sø	Uke	Ma	Ti	On	To	Fr	Lø	Sø	Uke	Ma	Ti	On	To	Fr	Lø	Sø
40			1	2	3	4	5	44						1	2	49	1	2	3	4	5	6	7
41	6	7	8	9	10	11	12	45	3	4	5	6	7	8	9	50	8	9	10	11	12	13	14
42	13	14	15	16	17	18	19	46	10	11	12	13	14	15	16	51	15	16	17	18	19	20	21
43	20	21	22	23	24	25	26	47	17	18	19	20	21	22	23	52	22	23	24	25	26	27	28
44	27	28	29	30	31			48	24	25	26	27	28	29	30		29	30	31				

Self-study

Part #1

Part #2

Part #3 (project)

Part Exam

Lab exercise

Lab Exercise #2.1 – Basic PLC programming

▼ Part II (PLC Software Development)
Lectures
 MAS418 - Lecture #2.1 A25.pdf
Lab exercises
 #2.1 - Basic PLC programming